Ministério da Educação
Secretaria de Educação Profissional e Tecnológica
Instituto Federal Catarinense
Câmpus Videira

GABRIEL NOGUEIRA

AGENTES AUTÔNOMOS BASEADOS EM MODELOS DE LINGUAGEM DE GRANDE ESCALA PARA IDENTIFICAÇÃO E CORREÇÃO DE INCONSISTÊNCIAS FISCAIS EM SISTEMAS ERP

Videira
2026

GABRIEL NOGUEIRA

AGENTES AUTÔNOMOS BASEADOS EM MODELOS DE LINGUAGEM DE GRANDE ESCALA PARA IDENTIFICAÇÃO E CORREÇÃO DE INCONSISTÊNCIAS FISCAIS EM SISTEMAS ERP

Trabalho de Conclusão de Curso apresentado ao Curso de graduação em Ciência da Computação do Instituto Federal Catarinense – Câmpus Videira para obtenção do título de bacharel em Ciência da Computação.
Orientador: Fabricio Bizotto

Videira
2026

GABRIEL NOGUEIRA

AGENTES AUTÔNOMOS BASEADOS EM MODELOS DE LINGUAGEM DE GRANDE ESCALA PARA IDENTIFICAÇÃO E CORREÇÃO DE INCONSISTÊNCIAS FISCAIS EM SISTEMAS ERP

Este Trabalho de Curso foi julgado adequado para a obtenção do título de Bacharel em Ciência da Computação e aprovado em sua forma final pelo curso de graduação em Ciência da Computação do Instituto Federal Catarinense – Campus Videira.

Videira (SC), dia de mês de ano

Fabricio Bizotto
Instituto Federal Catarinense – Campus Videira

BANCA EXAMINADORA

FULANO
Instituto Federal Catarinense – Campus Videira

BELTRANO
Instituto Federal Catarinense – Campus Videira

RESUMO

Inconsistências no cadastro fiscal de produtos em sistemas de gestão empresarial (ERPs), como a Nomenclatura Comum do Mercosul (NCM) vazia ou inexistente, levam à rejeição da Nota Fiscal Eletrônica pela Secretaria da Fazenda e geram demanda recorrente ao suporte técnico. Este trabalho desenvolve e avalia um agente baseado em Modelos de Linguagem de Grande Escala (LLMs), executado localmente via Ollama, que detecta produtos com NCM inválido no banco de dados Firebird do ERP Lux Auto e sugere a classificação correta a partir da descrição do produto, sob supervisão humana (*Human-in-the-Loop*). A detecção é determinística, por consulta à tabela NCM oficial; a sugestão combina uma busca lexical (SQLite FTS5 com BM25, sinônimos do WordNet e fusão de rankings) sobre o caminho hierárquico completo de cada código com duas escolhas do LLM restritas por esquema JSON, de modo que o modelo só pode responder com códigos existentes. As três opções mais prováveis são exibidas em uma interface Streamlit e só são gravadas no ERP após aprovação do operador. Foram comparados dois modelos abertos (Llama 3.1 8B e Qwen2.5 7B) e três arquiteturas (agente com chamada de ferramentas, busca seguida de escolha e navegação hierárquica nível a nível), usando conjuntos separados de desenvolvimento e de teste. A detecção identificou todos os produtos com NCM inválido do conjunto de teste. Na sugestão, a versão final, com o Qwen2.5 7B, acertou o código em primeiro lugar em 48,5% e entre as três opções em 63,6% dos 33 produtos de teste, levando cerca de 13 segundos por produto em uma GPU de 8 GB. A meta de 75% de acerto definida no projeto não foi atingida; a análise de erros mostra que a maior parte das falhas ocorre na escolha do item dentro da posição correta e em lacunas de vocabulário entre a descrição comercial e o texto oficial da tabela. Os resultados indicam que um LLM pequeno executado localmente é viável como assistente de classificação fiscal, desde que restrito a escolher entre candidatos verificados e subordinado à aprovação humana.

Palavras-chave: Modelos de Linguagem de Grande Escala. Classificação Fiscal. NCM. Geração Aumentada por Recuperação. *Human-in-the-Loop*. ERP.

ABSTRACT

Inconsistencies in the tax registration of products in Enterprise Resource Planning (ERP) systems, such as an empty or nonexistent Mercosur Common Nomenclature (NCM) code, cause Brazilian electronic invoices to be rejected by the tax authority and create recurring demand for technical support. This work develops and evaluates an agent based on Large Language Models (LLMs), run locally through Ollama, that detects products with an invalid NCM code in the Firebird database of the Lux Auto ERP and suggests the correct classification from the product description under human supervision (Human-in-the-Loop). Detection is deterministic, by lookup in the official NCM table; suggestion combines lexical retrieval (SQLite FTS5 with BM25, WordNet synonyms and rank fusion) over the full hierarchical path of each code with two LLM choices constrained by a JSON schema, so the model can only answer with existing codes. The three most likely options are shown in a Streamlit interface and are written to the ERP only after operator approval. Two open models (Llama 3.1 8B and Qwen2.5 7B) and three architectures (tool-calling agent, retrieve-then-choose, and level-by-level hierarchical navigation) were compared using separate development and test sets. Detection identified every product with an invalid NCM code in the test set. For suggestion, the final version with Qwen2.5 7B ranked the correct code first for 48.5% and among the three options for 63.6% of the 33 test products, taking about 13 seconds per product on an 8 GB GPU. The 75% accuracy target set in the project proposal was not met; error analysis shows that most failures come from choosing the item within the correct heading and from vocabulary gaps between commercial descriptions and the official table text. The results indicate that a small, locally run LLM is viable as a tax classification assistant, provided it is restricted to choosing among verified candidates and subordinate to human approval.

Keywords: Large Language Models. Tax Classification. NCM. Retrieval-Augmented Generation. Human-in-the-Loop. ERP.

LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxo de rejeição de NF-e e geração de demanda de suporte técnico 13
Figura 2 – Estrutura hierárquica de um código NCM 18
Figura 3 – Etapas do desenvolvimento do trabalho 25
Figura 4 – Arquitetura final do protótipo 29
Figura 5 – Etapas da sugestão de NCM na versão final 32
Figura 6 – Interface de supervisão com as opções sugeridas para um produto 36

LISTA DE QUADROS

Quadro 1 – Ferramentas e versões utilizadas 26
Quadro 2 – Versões do processo de sugestão avaliadas 37

LISTA DE TABELAS

Tabela 1 – Conjuntos de casos utilizados na avaliação 27
Tabela 2 – Acerto da opção única por modelo no conjunto dev15 39
Tabela 3 – Desempenho das versões do processo de sugestão com o Qwen2.5 7B . . . 40
Tabela 4 – Erros da versão final no conjunto de teste, por nível da hierarquia 41
Tabela 5 – Conjunto dev15 51
Tabela 6 – Conjunto dev 51
Tabela 7 – Conjunto de teste 53

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface*
BM25	*Best Matching 25* (função de ranqueamento)
CBS	Contribuição sobre Bens e Serviços
CFOP	Código Fiscal de Operações e Prestações
CST	Código de Situação Tributária
ERP	*Enterprise Resource Planning*
FTS	*Full-Text Search*
GPU	*Graphics Processing Unit*
HITL	*Human-in-the-Loop*
IBS	Imposto sobre Bens e Serviços
ICMS	Imposto sobre Circulação de Mercadorias e Serviços
JSON	*JavaScript Object Notation*
LGPD	Lei Geral de Proteção de Dados Pessoais
LLM	*Large Language Model*
MOC	Manual de Orientação do Contribuinte
NBS	Nomenclatura Brasileira de Serviços
NCM	Nomenclatura Comum do Mercosul
NF-e	Nota Fiscal Eletrônica
RAG	*Retrieval-Augmented Generation*
RRF	*Reciprocal Rank Fusion*
SEFAZ	Secretaria da Fazenda
SH	Sistema Harmonizado de Designação e de Codificação de Mercadorias
SQL	*Structured Query Language*
VRAM	*Video Random Access Memory*

SUMÁRIO

1 INTRODUÇÃO 11
1.1 Objetivos 12
1.1.1 Objetivo Geral 12
1.1.2 Objetivos Específicos 12
1.2 Justificativa 13
1.3 Estrutura do Trabalho 14
2 FUNDAMENTAÇÃO TEÓRICA 15
2.1 Sistemas ERP 15
2.1.1 Qualidade de Dados em Sistemas ERP 16
2.2 Nota Fiscal Eletrônica e Legislação Fiscal Brasileira 16
2.2.1 Nomenclatura Comum do Mercosul 17
2.3 Modelos de Linguagem de Grande Escala 18
2.3.1 Execução Local e Modelos Abertos 19
2.3.2 Saídas Estruturadas e Decodificação Restrita 20
2.4 Agentes e *Human-in-the-Loop* 20
2.5 Recuperação de Informação e Geração Aumentada por Recuperação 21
2.5.1 Busca Lexical e BM25 21
2.5.2 Fusão de Rankings 22
2.6 Streamlit 22
2.7 Integração de LLMs com Bancos de Dados Relacionais 22
2.8 Trabalhos Correlatos 23
3 METODOLOGIA 25
3.1 Etapas da Pesquisa 25
3.2 Ambiente Experimental 26
3.3 Conjuntos de Casos 26
3.4 Métricas 27
3.5 Protocolo de Avaliação 28
4 DESENVOLVIMENTO DO PROTÓTIPO 29
4.1 Visão Geral da Arquitetura 29
4.2 Base Local da Tabela NCM 30

4.3 Detecção de Produtos com NCM Inválido 31
4.4 Sugestão da Classificação 31
4.4.1 Memória de Classificações 32
4.4.2 Análise do Produto 33
4.4.3 Busca de Candidatos 33
4.4.4 Escolhas Restritas do Modelo 34
4.5 Execução em Segundo Plano e Notificação 35
4.6 Interface de Supervisão 35
4.7 Evolução das Versões 37
5 RESULTADOS E DISCUSSÃO 39
5.1 Detecção 39
5.2 Comparação entre Modelos de Linguagem 39
5.3 Comparação entre Versões do Processo de Sugestão 40
5.4 Análise dos Erros da Versão Final 41
5.5 Atendimento às Metas do Projeto 42
5.6 Discussão 42
5.6.1 Limitações e Ameaças à Validade 43
6 CONCLUSÃO 45
6.1 Trabalhos Futuros 45
REFERÊNCIAS 47
APÊNDICE A Conjuntos de Casos 51
APÊNDICE B *Prompts* da Versão Final 55

1 INTRODUÇÃO

Nos dias atuais, sistemas de gestão empresarial (ERPs) dependem fortemente da inserção manual de dados cadastrais para o seu pleno funcionamento. Essa dependência operacional gera gargalos significativos, uma vez que a interação humana está sujeita a falhas de digitação ou interpretação (Mehta, 2025). Tais inconsistências em bancos de dados relacionais dividem-se, em geral, em duas categorias. A primeira envolve erros de preenchimento que não impedem o funcionamento básico do sistema, como a inserção de um número de telefone no campo destinado ao endereço. A segunda, e mais crítica, engloba informações que afetam diretamente o ecossistema de integrações do software. Um exemplo claro ocorre no módulo fiscal: quando o cadastro de um cliente apresenta divergências entre a Inscrição Estadual e o Indicador de Contribuinte, ou há a omissão e preenchimento incorreto de códigos obrigatórios como NCM (Nomenclatura Comum do Mercosul), CST (Código de Situação Tributária) e CFOP (Código Fiscal de Operações e Prestações), a emissão da Nota Fiscal Eletrônica (NF-e) é inviabilizada e sumariamente rejeitada pela Secretaria da Fazenda (SEFAZ) (ENCAT; SEFAZ, 2020).

Essas rejeições paralisam o faturamento das empresas e geram uma sobrecarga imediata no setor de suporte técnico, exigindo intervenção manual para correções que, na maioria das vezes, seguem padrões lógicos e regulatórios bem definidos. Nesse cenário, os agentes baseados em Modelos de Linguagem de Grande Escala (LLMs) surgem como uma alternativa tecnológica promissora (Wang *et al.*, 2024). Ao combinar o processamento de linguagem natural com regras de negócio predefinidas, esses agentes podem identificar anomalias de forma proativa e formular propostas de correção (Roychowdhury *et al.*, 2025). Para garantir a segurança e a integridade da operação no banco de dados, a atuação do agente ocorre de forma supervisionada (*Human-in-the-Loop*): a correção proposta aguarda a validação e aprovação de um operador humano antes de ser executada (Cheng; Cheng, 2026).

Dentre as inconsistências fiscais, a classificação do produto pela NCM se destaca por exigir interpretação. Verificar se um código existe é uma consulta simples; descobrir qual código corresponde a uma descrição como “PARACHOQUE DIANTEIRO GOL” ou “AZULEJO DE BANHEIRO” exige conhecer uma tabela com mais de dez mil itens, organizada hierarquicamente e redigida em linguagem técnica que raramente coincide com o nome comercial do produto (BRASIL. Receita Federal, 2026). Por esse motivo, este trabalho concentra a implementação e a avaliação na NCM do cadastro de produtos, deixando as demais categorias

levantadas no projeto (CST, CFOP e Indicador de Contribuinte) como continuidade natural da arquitetura proposta.

Diante deste contexto, o presente trabalho busca responder à seguinte pergunta de pesquisa: é possível integrar um agente baseado em LLM a um banco de dados relacional para mitigar problemas cadastrais recorrentes, identificando inconsistências fiscais e propondo correções de forma eficiente sob supervisão humana?

1.1 OBJETIVOS

1.1.1 Objetivo Geral

Desenvolver e avaliar um agente baseado em Modelos de Linguagem de Grande Escala, executado localmente e integrado ao banco de dados relacional do ERP Lux Auto, capaz de identificar produtos com classificação fiscal (NCM) inválida e propor a classificação correta a partir da descrição do produto, com aprovação humana obrigatória antes de qualquer alteração no banco de dados.

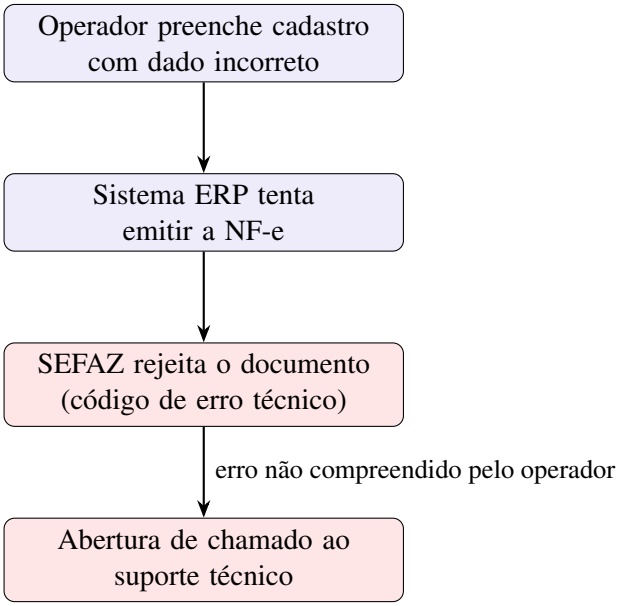
1.1.2 Objetivos Específicos

- Mapear as inconsistências fiscais mais recorrentes no cadastro do ERP Lux Auto que impedem a emissão de NF-e e delimitar a classificação NCM de produtos como foco da implementação;
- Estruturar a tabela NCM oficial em uma base local consultável, que permita validar códigos e recuperar candidatos a partir de descrições livres sem enviar a tabela inteira ao modelo de linguagem;
- Projetar e implementar o agente, definindo o fluxo de detecção, sugestão de correção, notificação e execução supervisionada por meio de uma interface independente do ERP;
- Construir conjuntos de casos separados para desenvolvimento e para teste, com a classificação correta de cada produto conferida na tabela oficial;
- Avaliar o agente quanto à taxa de detecção, à taxa de acerto das sugestões e ao tempo de resposta, comparando modelos de linguagem abertos e arquiteturas alternativas de sugestão.

1.2 JUSTIFICATIVA

A gestão fiscal em sistemas ERP representa um dos principais gargalos operacionais em pequenas e médias empresas do setor automotivo. Inconsistências cadastrais, como divergências entre a Inscrição Estadual e o Indicador de Contribuinte e o preenchimento incorreto de códigos NCM, CST e CFOP, comprometem diretamente a emissão de NF-e, interrompendo o fluxo de faturamento. Problemas de qualidade de dados são reconhecidos como fonte recorrente de dificuldades na operação de ERPs (Xu *et al.*, 2002), e, na experiência do autor no suporte técnico do Lux Auto, erros cadastrais de natureza fiscal respondem por parte expressiva dos chamados abertos pelos clientes. A Figura 1 sintetiza o fluxo que origina essa demanda.

Figura 1 – Fluxo de rejeição de NF-e e geração de demanda de suporte técnico



Fonte: O autor (2026).

Embora os próprios sistemas ERP sinalizem esses erros ao usuário, a baixa familiaridade dos operadores com terminologia fiscal faz com que as mensagens de erro sejam frequentemente ignoradas, resultando em chamados de suporte para problemas que já foram identificados pelo sistema, mas não compreendidos pelo usuário. Esse comportamento evidencia que a solução não passa apenas por detectar o erro, mas por interpretá-lo e propor uma correção acessível e supervisionada, de preferência no momento do cadastro, antes que a nota seja emitida.

Nesse contexto, abordagens baseadas apenas em regras fixas mostram-se insuficientes para cobrir a complexidade do domínio fiscal brasileiro (Nay *et al.*, 2023). Uma regra consegue dizer que o NCM “00000000” é inválido, mas não consegue dizer que um “PARA-

CHOQUE" deve ser classificado em "8708.10.00 – Para-choques e suas partes": isso exige interpretar a descrição do produto e relacioná-la ao texto da tabela oficial, tarefa em que os LLMs têm demonstrado bom desempenho (Gholamian *et al.*, 2024). Ao mesmo tempo, o risco de alucinação desses modelos (Huang *et al.*, 2025) torna indispensável que a resposta seja verificável e que a decisão final permaneça com um humano (Natarajan *et al.*, 2025). Investigar como equilibrar essas duas necessidades com um modelo pequeno, executado localmente e sem custo de API, é a contribuição prática deste trabalho.

1.3 ESTRUTURA DO TRABALHO

O Capítulo 2 apresenta a fundamentação teórica: sistemas ERP e qualidade de dados, a NF-e e a estrutura da NCM, os LLMs, as técnicas de recuperação de informação empregadas e os trabalhos correlatos. O Capítulo 3 descreve a metodologia, incluindo o ambiente experimental, os conjuntos de casos e as métricas. O Capítulo 4 detalha a arquitetura e a implementação do protótipo, bem como as versões testadas e descartadas ao longo do desenvolvimento. O Capítulo 5 apresenta e discute os resultados, e o Capítulo 6 traz as conclusões e os trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, são apresentados os conceitos e tecnologias que embasam o desenvolvimento deste trabalho. São abordados o funcionamento dos sistemas ERP e os desafios de qualidade de dados, as exigências da legislação fiscal brasileira para emissão de Notas Fiscais Eletrônicas e a estrutura da NCM, os fundamentos dos Modelos de Linguagem de Grande Escala, a abordagem de agentes com supervisão humana, as técnicas de recuperação de informação empregadas, as ferramentas de desenvolvimento adotadas e, por fim, os trabalhos correlatos identificados na literatura.

2.1 SISTEMAS ERP

Os sistemas de gestão empresarial, conhecidos pela sigla ERP (do inglês *Enterprise Resource Planning*), são plataformas de software integradas que centralizam e automatizam os principais processos organizacionais em um único ambiente (Arora; Erturk, 2017). Esses sistemas abrangem módulos como financeiro, fiscal, controle de estoque, vendas e relacionamento com clientes, permitindo que informações fluam entre os departamentos de forma padronizada. Sua adoção por pequenas e médias empresas tem crescido de forma expressiva nas últimas décadas, impulsionada pela necessidade de maior controle operacional e pela exigência de conformidade com normas regulatórias cada vez mais rigorosas (Laukkanen; Sarpola; Hallikainen, 2007; van Everdingen; van Hillegersberg; Waarts, 2000).

No setor automotivo brasileiro, especificamente em oficinas mecânicas, os sistemas ERP assumem papel estratégico. Dados do Anuário da Indústria de Reparação de Veículos indicam que o Brasil possui mais de 119 mil estabelecimentos do ramo automotivo (SINDIREPA BRASIL, 2023), o que evidencia a dimensão do mercado e a necessidade de eficiência operacional nesses negócios. Nesse contexto, o ERP integra o controle de ordens de serviço, a gestão de peças e insumos, o cadastro de clientes e fornecedores e a emissão de documentos fiscais em um único fluxo, tornando a consistência dos dados cadastrais um requisito não apenas técnico, mas legal.

Um exemplo prático dessa realidade é o sistema ERP Lux Auto, voltado ao segmento de oficinas mecânicas, desenvolvido em Delphi e com banco de dados Firebird. O sistema contempla o registro de produtos, o controle de serviços prestados em veículos, cadastros gerais de pessoas e fornecedores, além de módulos de contas a receber e a pagar. Embora ofereça uma

interface funcional para usuários experientes, novos clientes frequentemente enfrentam dificuldades nos primeiros meses de uso (Chimpiri, 2024), e é no módulo fiscal que os problemas mais críticos surgem, pois as validações impostas pelos órgãos governamentais são rígidas e intolerantes a erros de formatação ou classificação.

2.1.1 Qualidade de Dados em Sistemas ERP

A qualidade dos dados armazenados em sistemas ERP é fator determinante para a confiabilidade dos processos organizacionais (Xu *et al.*, 2002). Inconsistências cadastrais, sejam elas provenientes de erros de digitação, falta de padronização ou desconhecimento técnico dos operadores, comprometem a integridade das informações e geram um efeito cascata em todos os processos dependentes desses dados (Chimpiri, 2024). No contexto fiscal, essas inconsistências assumem caráter crítico: campos como Inscrição Estadual, Indicador de Contribuinte, NCM, CST e CFOP são validados por órgãos reguladores no momento da emissão de documentos fiscais eletrônicos (ENCAT; SEFAZ, 2020). A presença de dados incorretos ou em branco nesses campos inviabiliza o faturamento da empresa emissora, gerando demanda urgente e recorrente ao suporte técnico (Mehta, 2025).

Os ERPs atendem negócios dos mais variados perfis, e no nicho de oficinas mecânicas essa discrepância de conhecimento tecnológico fica ainda mais evidente. O sistema precisa atender desde profissionais qualificados em rotinas fiscais até usuários com pouca familiaridade com computação básica. Na experiência do autor com o Lux Auto, a maioria dos gargalos operacionais recai sobre usuários que não conseguem, por conta própria, ajustar campos como NCM ou Indicador de Contribuinte a partir de um código de rejeição devolvido pela SEFAZ. Nesse cenário, mecanismos de validação e correção preventiva, que atuem no momento do cadastro, tornam-se essenciais para manter a operação do negócio ativa (Schelter *et al.*, 2018).

2.2 NOTA FISCAL ELETRÔNICA E LEGISLAÇÃO FISCAL BRASILEIRA

A Nota Fiscal Eletrônica (NF-e) é um documento fiscal digital instituído pelo Ajuste SINIEF 07/2005, que substituiu o modelo em papel e conferiu maior agilidade e controle às operações comerciais (BRASIL, 2005). Sua emissão é regulamentada pela SEFAZ e está condicionada à validação em tempo real por meio de serviços web governamentais, que verificam centenas de regras de negócio antes de autorizar o documento (ENCAT; SEFAZ, 2020).

Entre os principais campos obrigatórios validados pela SEFAZ, destacam-se o Có-

digo de Situação Tributária (CST), que define o tratamento do ICMS aplicável à operação; o Código Fiscal de Operações e Prestações (CFOP), que classifica a natureza da operação comercial; e a Nomenclatura Comum do Mercosul (NCM), que identifica a classificação fiscal do produto e determina suas alíquotas tributárias (SEFAZ-PE, 2016). Além desses, o documento exige a descrição detalhada dos produtos, tributos como IPI, PIS, COFINS e ISS e os dados completos do emitente e do destinatário (ENCAT; SEFAZ, 2020). Uma única divergência nesses campos resulta na rejeição do documento, interrompendo o fluxo de faturamento da empresa emissora (BRASIL, 2005).

Com a reforma tributária em curso no Brasil (BRASIL, 2023), que substitui progressivamente os tributos vigentes pelo IBS (Imposto sobre Bens e Serviços) e pela CBS (Contribuição sobre Bens e Serviços), os documentos fiscais eletrônicos passam a exigir novos campos e regras de classificação durante o período de transição. Para sistemas ERP, isso amplia o espaço de possíveis inconsistências cadastrais, tornando ainda mais improvável que um sistema baseado apenas em regras fixas acompanhe todas as combinações válidas. Abordagens que combinam LLMs com representações formais das regras tributárias têm sido propostas como forma de manter a confiabilidade do raciocínio fiscal diante de mudanças regulatórias frequentes (Jurayj; Holzenberger; Van Durme, 2026).

2.2.1 Nomenclatura Comum do Mercosul

A NCM é o sistema de classificação de mercadorias adotado pelos países do Mercosul, utilizado tanto no comércio exterior quanto nas operações internas, como a emissão de NF-e (BRASIL. Receita Federal, 2026). Ela tem por base o Sistema Harmonizado de Designação e de Codificação de Mercadorias (SH), mantido pela Organização Mundial das Aduanas, que padroniza internacionalmente os seis primeiros dígitos do código (WORLD CUSTOMS ORGANIZATION, 2026). A NCM acrescenta dois dígitos próprios, formando um código de oito dígitos organizado de forma hierárquica, como ilustra a Figura 2: os dois primeiros identificam o Capítulo; os quatro primeiros, a Posição; o quinto e o sexto, as Subposições; e o sétimo e o oitavo, o Item e o Subitem. Somente o código completo de oito dígitos é válido para o cadastro de um produto.

Figura 2 – Estrutura hierárquica de um código NCM

87 Capítulo: Veículos automóveis, tratores, ciclos e outros veículos terrestres, suas partes e acessórios

87.08 Posição: Partes e acessórios dos veículos automóveis das posições 87.01 a 87.05

8708.10.00 Item: Para-choques e suas partes (único nível válido no cadastro)

Fonte: O autor (2026), com base na Tabela NCM vigente.

A tabela vigente utilizada neste trabalho possui 96 capítulos e 10.504 códigos de oito dígitos. Duas características dessa tabela são centrais para o problema estudado. A primeira é que a descrição própria de um item frequentemente só faz sentido junto com os níveis superiores: o item 4011.10.00 é descrito apenas como "Do tipo utilizado em automóveis de passageiros", e é o cabeçalho da posição 40.11 que informa tratar-se de pneumáticos novos de borracha. A segunda é que o texto oficial usa linguagem técnica, distante do nome comercial do produto: "sabonete" aparece como "sabões", "cadeira" como "assentos" e "notebook" como "máquinas automáticas para processamento de dados". Além disso, a classificação segue as Regras Gerais de Interpretação do SH, segundo as quais a mercadoria é classificada, em primeiro lugar, pelos textos das posições e das notas de seção e de capítulo, e não pelo título do capítulo (WORLD CUSTOMS ORGANIZATION, 2026).

Cabe observar que, no Lux Auto, o mesmo campo do cadastro guarda também códigos de serviços, que seguem outra classificação, a Nomenclatura Brasileira de Serviços (NBS). Por isso, a validação da NCM deve se restringir aos itens cadastrados como mercadoria, sob pena de apontar como erro um código de serviço legítimo.

2.3 MODELOS DE LINGUAGEM DE GRANDE ESCALA

Os Modelos de Linguagem de Grande Escala, conhecidos pela sigla LLM (do inglês *Large Language Model*), são sistemas de inteligência artificial baseados na arquitetura *Transformer*, proposta por Vaswani *et al.* (2017). Essa arquitetura utiliza mecanismos de autoatenção (*self-attention*) que permitem ao modelo processar sequências de texto de forma paralela, identificando relações contextuais entre palavras independentemente de sua posição na sentença (Shao *et al.*, 2024). Os LLMs são treinados em volumes massivos de dados textuais, o que lhes confere a capacidade não apenas de gerar texto coerente, mas de interpretar contextos comple-

xos e inferir intenções (Douglas, 2023).

No domínio fiscal, essa capacidade se mostra especialmente relevante. Diferentemente de abordagens baseadas em regras fixas (*hard-coded*), que falham ao tentar cobrir a totalidade de exceções da legislação tributária, os LLMs conseguem interpretar situações ambíguas e formular respostas adaptadas ao contexto (Nay *et al.*, 2023). A classificação de um produto pelo código NCM, por exemplo, exige a interpretação de descrições textuais e o cruzamento com tabelas regulatórias, tarefa na qual os LLMs têm demonstrado desempenho superior ao de sistemas baseados em regras estáticas (Gholamian *et al.*, 2024).

Entretanto, o uso de LLMs em ambientes de produção esbarra em uma limitação crítica: as alucinações, isto é, respostas incorretas geradas pelo modelo com alto grau de confiança (Huang *et al.*, 2025). No ambiente fiscal, uma classificação tributária equivocada pode resultar em recolhimento indevido de impostos e em penalidades. Por essa razão, a literatura recomenda que aplicações de LLMs em domínios sensíveis não busquem a autonomia total do modelo, mas o utilizem como ferramenta de análise e proposição, subordinada à validação humana (Natarajan *et al.*, 2025).

2.3.1 Execução Local e Modelos Abertos

Modelos proprietários oferecidos via API cobram por volume de *tokens* processados, e esse custo recorrente pode tornar a operação contínua inviável a partir de determinados volumes de uso (Pan *et al.*, 2025). Uma alternativa é a execução local de modelos de código aberto por meio de ferramentas como o Ollama, que permite rodar LLMs diretamente na máquina, sem dependência de serviços externos (Ollama, 2023). Técnicas de otimização de inferência permitem reduzir significativamente os requisitos computacionais desses modelos (Zhou *et al.*, 2024); entre elas, destaca-se a quantização, que reduz a precisão numérica dos parâmetros (tipicamente de 16 para 4 ou 5 *bits*), seja por treinamento com quantização nativa (Liu *et al.*, 2024), seja por quantização pós-treinamento, como nos formatos do padrão GGUF adotado pelo Ollama (Ollama, 2023). Com essa técnica, modelos de 7 a 8 bilhões de parâmetros podem operar com menos de 6 GB de memória de vídeo (VRAM), o que viabiliza seu uso em computadores convencionais e mantém os dados da empresa no próprio ambiente.

Duas famílias de modelos abertos foram utilizadas neste trabalho. A família Llama, da Meta, cuja versão 3.1 com 8 bilhões de parâmetros é amplamente utilizada como referência em execução local (Grattafiori *et al.*, 2024), e a família Qwen2.5, da Alibaba, que inclui um

modelo de 7 bilhões de parâmetros treinado com ênfase em suporte multilíngue e na geração de saídas estruturadas (Qwen Team, 2024).

2.3.2 Saídas Estruturadas e Decodificação Restrita

Um LLM gera texto escolhendo, a cada passo, o próximo *token* a partir de uma distribuição de probabilidades sobre todo o vocabulário. Na decodificação restrita (*constrained decoding* ou *guided generation*), essa escolha é limitada aos *tokens* que mantêm a saída válida segundo uma gramática formal, como uma expressão regular ou um esquema JSON (Willard; Louf, 2023). Na prática, isso garante que a resposta do modelo possa ser lida por um programa e, quando o esquema enumera os valores permitidos para um campo, que o modelo só consiga responder com um desses valores. O Ollama oferece esse recurso ao aceitar um esquema JSON como formato obrigatório da resposta (Ollama, 2023). Para a classificação fiscal, isso significa que o modelo pode ser impedido, por construção, de inventar um código NCM inexistente: ele só pode escolher entre os códigos apresentados.

2.4 AGENTES E *HUMAN-IN-THE-LOOP*

O avanço dos LLMs permitiu o surgimento de uma nova classe de sistemas: os agentes. Diferentemente de um LLM utilizado de forma isolada, que apenas recebe um texto e gera uma resposta, um agente é um sistema modular no qual o modelo de linguagem atua como motor de raciocínio, recebendo acesso a ferramentas externas para perceber o ambiente e interagir com ele (Wang *et al.*, 2024). Essas ferramentas podem incluir acesso a bancos de dados, APIs, mecanismos de busca e execução de código (Qu *et al.*, 2024).

No contexto de correção de bases de dados, um agente é capaz de: (1) conectar-se a um banco de dados relacional; (2) identificar registros com anomalias com base em regras de domínio; (3) formular uma proposta de correção; e (4) apresentar essa proposta ao operador humano para validação (Roychowdhury *et al.*, 2025). Entretanto, em domínios críticos como o fiscal e o financeiro, a delegação total de decisões a uma inteligência artificial apresenta riscos sistêmicos inaceitáveis (Pöhler *et al.*, 2025).

Para mitigar esse risco, a literatura recomenda a adoção do paradigma *Human-in-the-Loop* (HITL), no qual o fluxo de automação é intencionalmente interrompido antes de qualquer alteração permanente no banco de dados: o agente elabora o diagnóstico e formula a proposta de correção, mas aguarda a aprovação de um operador humano antes da execução

(Cheng; Cheng, 2026). Essa exigência se alinha a modelos mais amplos de segurança e responsabilização de agentes de IA, que defendem a validação progressiva da autonomia como condição para a confiabilidade do sistema (Cheng; Cheng; Siu, 2025). A abordagem também privilegia a acessibilidade, na medida em que a proposta pode ser apresentada em linguagem simples, sem expor termos técnicos ao usuário final (Madicharla, 2025).

2.5 RECUPERAÇÃO DE INFORMAÇÃO E GERAÇÃO AUMENTADA POR RECUPERAÇÃO

Um LLM não conhece de forma confiável o conteúdo atualizado de uma tabela regulatória com milhares de linhas, e a tabela inteira não cabe no contexto de um modelo pequeno. A Geração Aumentada por Recuperação (RAG, do inglês *Retrieval-Augmented Generation*) contorna esse limite: antes de gerar a resposta, um componente de busca recupera de uma base externa apenas os trechos relevantes para a pergunta, que são então entregues ao modelo junto com a instrução (Lewis *et al.*, 2020). Essa capacidade é especialmente útil em domínios regulatórios, nos quais a resposta correta depende de documentos que não fazem parte do treinamento do modelo (Kim; Min, 2025).

2.5.1 Busca Lexical e BM25

A forma mais tradicional de recuperação é a busca lexical, que localiza documentos que contêm as palavras da consulta por meio de um índice invertido. Para ordenar os resultados, a função de ranqueamento BM25 dá a cada documento uma pontuação que cresce com a quantidade de palavras da consulta presentes nele e com a raridade dessas palavras na coleção, compensando o tamanho do documento (Robertson; Zaragoza, 2009). Assim, termos raros, como “para-choques”, pesam mais do que termos comuns, como “outros”. O SQLite implementa essa função em sua extensão de busca textual FTS5, que permite ainda atribuir pesos diferentes a cada coluna indexada e normalizar acentos (SQLite, 2024).

A principal limitação da busca lexical é depender de palavras em comum entre a consulta e o documento. Para reduzir esse problema, é possível expandir a consulta com sinônimos obtidos de uma base lexical como a WordNet, que organiza palavras em conjuntos de sinônimos (Miller, 1995). Para o português, a OpenWordNet-PT oferece uma base desse tipo (Paiva; Rademaker; Melo, 2012), acessível em Python pela biblioteca NLTK (Bird; Klein; Loper, 2009).

2.5.2 Fusão de Rankings

Quando a mesma necessidade de informação é expressa de mais de uma forma, por exemplo pela descrição original de um produto e por uma reescrita em linguagem técnica, cada consulta produz uma lista ordenada diferente. A fusão por posição recíproca (RRF, do inglês *Reciprocal Rank Fusion*) combina essas listas somando, para cada documento d, o inverso de sua posição em cada lista $r \in R$ (Cormack; Clarke; Büttcher, 2009):

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)} \tag{2.1}$$

em que $r(d)$ é a posição de d na lista r e k é uma constante (tipicamente 60) que atenua a diferença entre as primeiras posições. O método não exige que as pontuações das listas sejam comparáveis entre si e é simples de implementar.

2.6 STREAMLIT

O Streamlit é um *framework* de código aberto em Python que permite a criação rápida de aplicações web interativas, amplamente utilizado no ecossistema de ciência de dados e inteligência artificial (Streamlit Inc., 2024). Sua principal vantagem reside na capacidade de construir interfaces conectadas a rotinas de IA sem a necessidade de conhecimento em HTML, CSS ou JavaScript. Por operar como uma aplicação independente (*stand-alone*) que se conecta diretamente a um banco de dados, sem exigir alterações no código-fonte do sistema que o origina, o Streamlit é adequado para a construção de camadas de supervisão desacopladas de sistemas legados (Madicharla, 2025; Ruiz Gómez *et al.*, 2025).

2.7 INTEGRAÇÃO DE LLMS COM BANCOS DE DADOS RELACIONAIS

A integração entre LLMs e bancos de dados relacionais é uma das áreas mais ativas de pesquisa em sistemas inteligentes aplicados (Hong *et al.*, 2024). A abordagem mais difundida, conhecida como *Text-to-SQL*, permite que o modelo interprete instruções em linguagem natural e as converta em consultas SQL executáveis. Tipicamente, o modelo recebe a estrutura do banco (DDL, do inglês *Data Definition Language*) e, a partir dela, gera a consulta correspondente a uma pergunta do usuário (Ruiz Gómez *et al.*, 2025). Frameworks como o LangChain fornecem abstrações para essa integração, gerenciando a conexão entre o modelo, as fontes de dados e as ferramentas disponíveis ao agente (Chase, 2022).

O Firebird é um sistema de gerenciamento de banco de dados relacional de código aberto que surgiu como *fork* do InterBase 6.0 da Borland em 2000, mantendo forte integração com o ecossistema Delphi (Firebird Project, 2024); por isso, é o banco adotado por muitos ERPs desenvolvidos nessa linguagem, entre eles o Lux Auto. Por questões de segurança, recomenda-se que a camada de integração imponha restrições (*guardrails*) que bloqueiem comandos destrutivos como `DROP` e `DELETE`, restringindo o agente a operações de leitura durante a fase de diagnóstico (Roychowdhury *et al.*, 2025).

2.8 TRABALHOS CORRELATOS

A aplicação de agentes baseados em LLMs para manipulação e qualidade de dados em ambientes empresariais é um campo de pesquisa em rápida expansão. A seguir, são apresentados os trabalhos que se relacionam diretamente com a proposta deste TCC.

Ruiz Gómez *et al.* (2025) apresentam um agente LLM capaz de interagir com um sistema ERP industrial por meio de linguagem natural, traduzindo consultas em instruções SQL executáveis por meio de uma arquitetura com dois agentes. O trabalho demonstra a viabilidade técnica da integração entre LLMs e sistemas ERP complexos, utilizando o LangChain como camada de orquestração, e comprova que a abordagem consegue operar sobre bases de dados reais de produção.

Bendinelli, Dox e Holz (2025) investigam o uso de agentes LLM para limpeza de *datasets* tabulares, demonstrando que esses modelos são capazes de identificar e corrigir entradas incorretas ao explorar informações contextuais de outras colunas do mesmo registro, com taxas de acerto superiores às de métodos heurísticos tradicionais.

Gholamian *et al.* (2024) investigam a classificação de produtos com LLMs em cenários de comércio e de conformidade regulatória, com atenção à robustez dos modelos diante de descrições de produtos incompletas ou ruidosas, característica também presente nos cadastros de ERP.

Por fim, Kim e Min (2025) propõem o QA-RAG, um sistema que integra IA generativa e recuperação de documentos para automação de processos de conformidade regulatória farmacêutica. O trabalho demonstra que a recuperação de trechos relevantes de documentos regulatórios, entregues ao modelo no momento da resposta, permite tratar requisitos complexos de forma estruturada, mesmo princípio que este trabalho aplica à tabela NCM.

Em conjunto, esses trabalhos demonstram que a integração de LLMs com sistemas

de gerenciamento de dados empresariais e com documentos regulatórios é tecnicamente viável. Entretanto, nenhum deles aborda o domínio específico da legislação fiscal brasileira nem as particularidades das validações impostas pela SEFAZ na emissão de NF-e. Esta proposta se diferencia por três aspectos: o recorte no nicho fiscal-tributário brasileiro; a integração direta com um sistema ERP real do setor automotivo nacional; e a execução inteiramente local, com um modelo pequeno, em hardware convencional.

3 METODOLOGIA

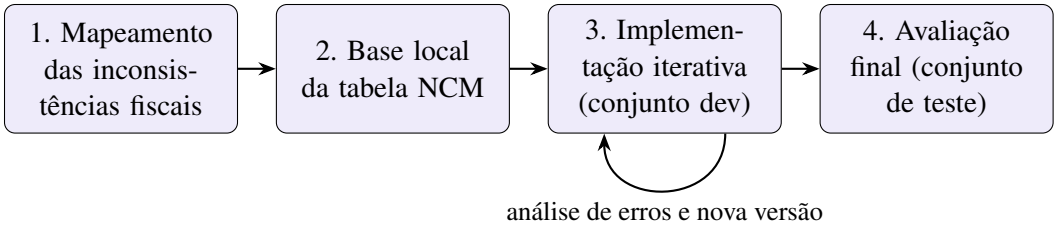
Este trabalho caracteriza-se como uma pesquisa aplicada, de abordagem quantitativa e natureza experimental, estruturada na forma de estudo de caso no sistema ERP Lux Auto. O conhecimento prévio sobre o sistema e suas regras de negócio relatado neste trabalho origina-se da experiência profissional do autor no suporte técnico e no desenvolvimento do software. O objetivo é desenvolver um agente capaz de identificar e corrigir inconsistências de classificação fiscal no cadastro de produtos e medir, de forma reprodutível, o quanto suas sugestões acertam.

3.1 ETAPAS DA PESQUISA

O trabalho foi conduzido em quatro etapas, sintetizadas na Figura 3. A primeira consistiu no mapeamento das inconsistências fiscais que mais geram rejeição de NF-e no Lux Auto. Foram levantadas quatro categorias: divergência entre Inscrição Estadual e Indicador de Contribuinte, classificação incorreta de NCM, aplicação equivocada de CST e preenchimento incorreto de CFOP. Dentre elas, a classificação NCM foi escolhida como foco da implementação, por ser a que mais depende de interpretação de texto e, portanto, a que melhor exercita a contribuição esperada de um LLM; as demais categorias são discutidas como trabalhos futuros.

A segunda etapa compreendeu a construção de uma base local com a tabela NCM vigente, a partir do arquivo oficial em PDF, de forma que a validação de códigos e a busca de candidatos não dependessem do modelo de linguagem. A terceira etapa consistiu no projeto e na implementação do protótipo, conduzidos de forma iterativa: cada versão foi avaliada em um conjunto de desenvolvimento, e os erros observados orientaram a versão seguinte. A quarta etapa foi a avaliação final das versões candidatas em um conjunto de teste separado, não utilizado nos ajustes.

Figura 3 – Etapas do desenvolvimento do trabalho



Fonte: O autor (2026).

3.2 AMBIENTE EXPERIMENTAL

Todos os experimentos foram executados localmente, em um notebook com processador AMD Ryzen AI 7 350, 31 GB de memória RAM e placa de vídeo NVIDIA GeForce RTX 5060 Laptop com 8 GB de VRAM, sob o sistema operacional Windows 11. O protótipo foi desenvolvido em Python 3.13, e os modelos de linguagem foram servidos pelo Ollama, com quantização de 4 *bits* (formato Q4_K_M), temperatura zero e janela de contexto de 8.192 *tokens*. O Quadro 1 resume as ferramentas utilizadas.

Quadro 1 – Ferramentas e versões utilizadas

Ferramenta	**Versão**	**Uso no trabalho**
Python	3.13	Linguagem do protótipo
Ollama	0.34.2	Execução local dos modelos de linguagem
Qwen2.5 7B	Q4_K_M	Modelo de linguagem da versão final
Llama 3.1 8B	Q4_K_M	Modelo de linguagem comparado
LangChain (langchain-ollama)	1.1.0	Comunicação com o Ollama e saída estruturada
Firebird / firebird-driver	3.0 / 2.0.3	Banco de dados do ERP e acesso via Python
SQLite (FTS5)	3.50.4	Base local da tabela NCM e busca textual
NLTK (OpenWordNet-PT)	3.9.4	Sinônimos em português
pdfplumber	0.11.10	Extração da tabela NCM do PDF oficial
Streamlit	1.64.0	Interface de supervisão humana

Fonte: O autor (2026).

O banco de dados utilizado é uma instalação de desenvolvimento do Lux Auto, com a mesma estrutura das instalações dos clientes. Os produtos usados nos testes foram cadastrados nessa base com o NCM “00000000”, simulando o operador que salva o cadastro sem saber a classificação. Diferentemente do previsto no projeto, não foram utilizados registros reais de rejeição de clientes: a construção de um conjunto anonimizado em conformidade com a LGPD não coube no cronograma, e a opção por produtos fictícios, porém realistas e de setores variados, permitiu controlar a resposta correta de cada caso.

3.3 CONJUNTOS DE CASOS

Para medir o desempenho sem superestimá-lo, os casos foram divididos em conjuntos com papéis diferentes, prática comum em aprendizado de máquina. O conjunto de desenvolvimento é usado durante a construção do sistema: os erros observados nele orientam ajustes de *prompt*, de busca e de arquitetura, e por isso o resultado medido nele tende a ser otimista. O

conjunto de teste é reservado para a medição final de cada versão e não orienta nenhum ajuste. A Tabela 1 apresenta os três conjuntos utilizados, e o Apêndice A lista os produtos de cada um.

Tabela 1 – Conjuntos de casos utilizados na avaliação

Conjunto	Produtos	Composição	Papel na pesquisa
dev15	15	Peças automotivas e um azulejo, com variações de escrita	Primeiros ajustes e comparação entre modelos; resultado otimista
dev	41	14 setores, 13 produtos automotivos	Orientou as melhorias das versões finais
teste	33	17 setores, 4 produtos automotivos, cadastrados no ERP	Medição final de cada versão, sem ajustes

Fonte: O autor (2026).

Para cada produto, a classificação correta foi conferida pelo autor na tabela NCM oficial. Quando a descrição admite mais de um código (por exemplo, um tênis pode ser de borracha ou de outro material, ou um suco de uva pode ter valor Brix diferente), todos os códigos plausíveis foram aceitos como corretos. As descrições foram escritas como um operador as digitaria no ERP, em letras maiúsculas, com abreviações, unidades e, em alguns casos, erros de digitação reais (“GELADERA”, “FURADERA”, “AZULEIJO”).

3.4 MÉTRICAS

Seguindo o projeto, a avaliação considera duas dimensões principais, detecção e correção, complementadas por métricas que ajudam a localizar a origem dos erros:

- **Taxa de detecção**: proporção dos produtos com NCM inválido que foram apontados pelo sistema;
- **Acerto em 1º lugar**: proporção dos produtos em que a primeira opção sugerida é um código correto;
- **Acerto entre as 3 opções**: proporção dos produtos em que um código correto aparece entre as três opções apresentadas ao operador. Como a decisão final é humana, essa é a métrica que melhor representa a utilidade prática do sistema;
- **Capítulo correto**: proporção dos produtos em que o capítulo escolhido pelo modelo contém a resposta, o que permite separar erros de “que tipo de produto é este” de erros de escolha do item;

- **Tempo médio por produto**: tempo de ponta a ponta para gerar as sugestões de um produto;
- **Falhas de infraestrutura**: casos em que o servidor do modelo não respondeu, contabilizados à parte para não serem confundidos com erros de classificação.

O critério de sucesso definido no projeto foi uma taxa de detecção de pelo menos 85% e uma taxa de acerto das correções de pelo menos 75%. Cada execução da avaliação registra, para cada produto, as opções sugeridas, a resposta esperada, o capítulo escolhido, a interpretação feita pelo modelo e o tempo gasto, de forma que os resultados possam ser auditados e reproduzidos.

3.5 PROTOCOLO DE AVALIAÇÃO

Na avaliação, o sistema recebe somente a descrição do produto, exatamente como cadastrada. A memória de produtos parecidos, descrita no Capítulo 4, permanece ativa, mas é configurada para ignorar cadastros com descrição idêntica à do produto avaliado, evitando que um produto já corrigido anteriormente "entregue" a própria resposta. Os modelos são executados com temperatura zero, o que torna as respostas praticamente determinísticas. Todas as versões comparadas foram medidas nos mesmos conjuntos, na mesma máquina.

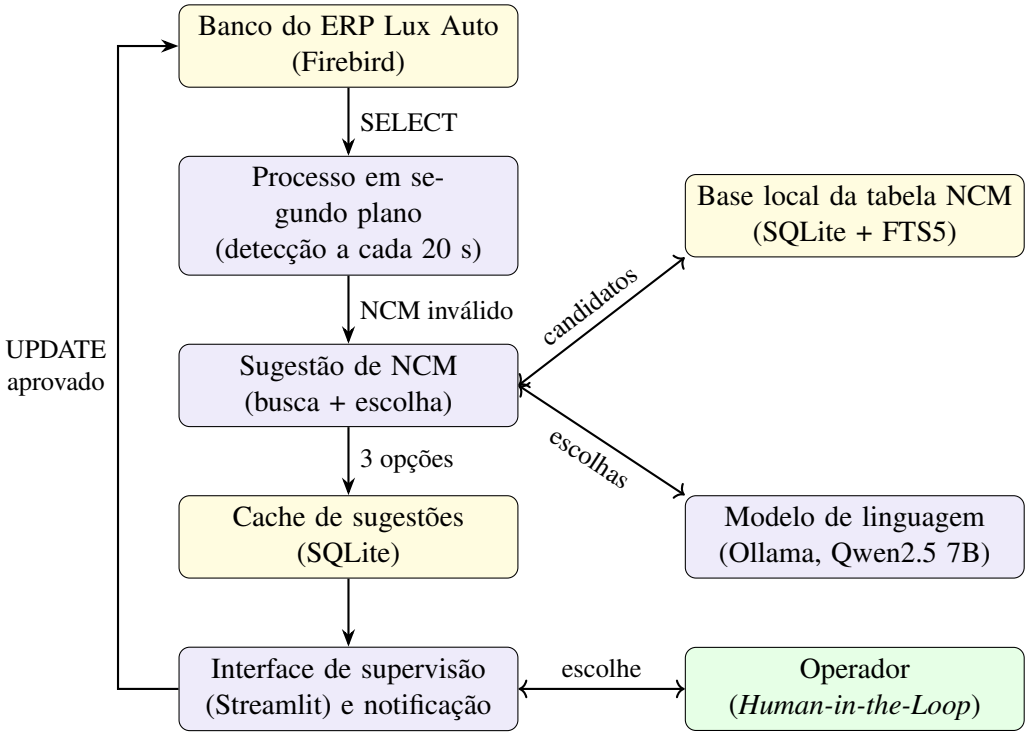
4 DESENVOLVIMENTO DO PROTÓTIPO

Este capítulo descreve a arquitetura e a implementação do protótipo. Primeiro, apresenta-se a visão geral da solução final; em seguida, cada componente é detalhado: a base local da tabela NCM, a detecção de produtos com NCM inválido, o processo de sugestão, a execução em segundo plano e a interface de supervisão. Ao final, descreve-se a evolução das versões testadas ao longo do trabalho, incluindo as que foram descartadas e os motivos.

4.1 VISÃO GERAL DA ARQUITETURA

A Figura 4 apresenta a arquitetura final. O protótipo é uma aplicação independente do ERP, que se conecta ao mesmo banco de dados Firebird e não exige nenhuma alteração no código-fonte do Lux Auto. Um processo em segundo plano consulta periodicamente o banco e identifica produtos com NCM inválido. Para cada um, o processo de sugestão consulta uma base local com a tabela NCM e o modelo de linguagem, executado pelo Ollama, e grava as três opções mais prováveis em um cache local. O operador é avisado por uma notificação do Windows e, na interface Streamlit, escolhe uma das opções; somente então o NCM é gravado no cadastro do produto.

Figura 4 – Arquitetura final do protótipo



Fonte: O autor (2026).

Em relação à arquitetura prevista no projeto, a principal mudança é o papel do modelo de linguagem. No projeto, o agente receberia a estrutura do banco e geraria consultas SQL a partir de instruções em linguagem natural (*Text-to-SQL*). Na solução final, todas as consultas SQL são fixas e parametrizadas, escritas no código, e o modelo não tem acesso ao banco: ele recebe apenas a descrição do produto e uma lista de códigos candidatos, e sua resposta é restrita, por esquema JSON, a escolher entre esses códigos. A seção 4.7 explica como se chegou a essa decisão. O fluxo de aprovação humana previsto no projeto foi mantido integralmente.

4.2 BASE LOCAL DA TABELA NCM

A tabela NCM vigente foi obtida em formato PDF, com 505 páginas, e extraída para um banco SQLite com a biblioteca pdfplumber, resultando em 15.141 linhas, que abrangem todos os níveis da hierarquia, dos capítulos aos itens. Durante a extração, observou-se que, em parte das linhas, uma camada de texto sobreposta no PDF fazia os caracteres da data de vigência se intercalarem com o final da descrição. Como a ordem relativa dos caracteres de cada campo se preserva nessa intercalação, um script de reparo separou os dígitos que formam a data do restante do texto e reconstruiu a descrição. Algumas descrições ainda apresentam pequenos defeitos herdados da extração, como palavras emendadas.

Essa base resolveu um problema prático observado no início do trabalho: tentar fornecer a tabela inteira a um modelo de linguagem excede qualquer janela de contexto viável. Com a base local, a validação de um código passa a ser uma consulta direta, e o modelo recebe apenas algumas dezenas de candidatos relevantes, no espírito da RAG descrita na seção 2.5.

Para a busca, foi construído um índice de texto completo (FTS5) somente sobre os 10.504 códigos de oito dígitos, já que apenas eles são válidos no cadastro. Como a descrição de um item frequentemente só faz sentido com os níveis superiores (seção 2.2.1), cada item foi indexado com três colunas: o texto próprio do item; o texto de todos os seus ancestrais (capítulo, posição e subposições); e uma versão do texto sem hífens. A primeira coluna recebe peso maior no BM25 (5,0, contra 1,0 dos ancestrais e 0,5 da versão sem hífen), de modo que um item cujo próprio texto corresponde à busca fique à frente de um item que só corresponde pelo cabeçalho. A coluna sem hífen permite que descrições digitadas sem hífen, como “PARACHOQUE”, encontrem “para-choques”. O índice ignora acentos, o que torna a busca tolerante a descrições digitadas sem acentuação.

4.3 DETECÇÃO DE PRODUTOS COM NCM INVÁLIDO

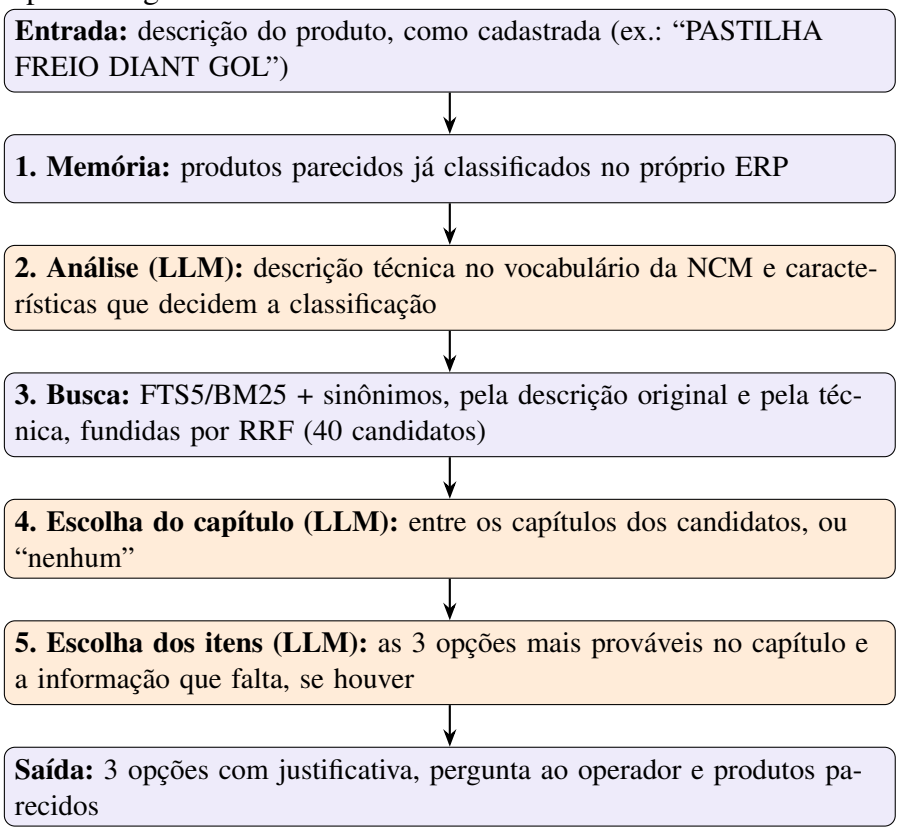
A detecção não utiliza o modelo de linguagem. Uma consulta somente leitura obtém todos os produtos cadastrados como mercadoria (campo `SERVICO = 'N'`), excluindo os serviços, cujo campo NCM guarda um código NBS (seção 2.2.1). Para cada produto, o código cadastrado é normalizado (o ERP grava os oito dígitos sem pontuação) e considerado inválido quando está vazio, não possui oito dígitos ou não existe na tabela vigente. Por ser uma regra determinística, a detecção não está sujeita a alucinação e executa em milissegundos.

Essa regra detecta códigos vazios, malformados ou inexistentes, que são os que causam rejeição imediata da NF-e por código inválido. Ela não detecta um produto cadastrado com um código válido, porém de outro produto, por exemplo um parafuso classificado como pneu. Esse tipo de erro exigiria comparar a descrição do produto com o texto do código cadastrado e é discutido como trabalho futuro.

4.4 SUGESTÃO DA CLASSIFICAÇÃO

O processo de sugestão recebe a descrição do produto e devolve até três códigos NCM, do mais para o menos provável, cada um com uma justificativa curta. A Figura 5 resume suas etapas, detalhadas a seguir.

Figura 5 – Etapas da sugestão de NCM na versão final



Fonte: O autor (2026). Em laranja, as etapas que consultam o modelo de linguagem.

4.4.1 Memória de Classificações

Em uma oficina, um produto novo quase sempre se parece com algum já cadastrado, como "PASTILHA FREIO DIANT GOL" e "PASTILHA FREIO TRAS PALIO". O NCM que a empresa já usa para o produto parecido é a pista mais forte disponível, pois reflete como aquela empresa classifica seus produtos. Por isso, a primeira etapa procura, entre os produtos do próprio ERP que já possuem NCM válido, aqueles com descrição semelhante. A semelhança entre os conjuntos de palavras A e B das duas descrições, sem acentos e sem palavras genéricas, é calculada como a média entre o coeficiente de Jaccard e a sobreposição relativa ao menor conjunto:

$$\mathrm{sim}(A,B) = \frac{1}{2}\,\frac{|A \cap B|}{|A \cup B|} + \frac{1}{2}\,\frac{|A \cap B|}{\min(|A|,|B|)} \tag{4.1}$$

O segundo termo foi incluído porque o Jaccard sozinho penaliza o caso mais comum em cadastros, em que uma descrição está contida na outra ("PARACHOQUE" e "PARACHOQUE DIANTEIRO GOL"). Produtos com semelhança a partir de 0,4 são considerados parecidos. O código do mais parecido é inserido no topo da lista de candidatos, e até três pro-

dutos parecidos são informados ao modelo como indício a ser conferido. Como as aprovações feitas na interface gravam o NCM no próprio cadastro, essa memória cresce com o uso, sem exigir nenhuma tabela adicional.

4.4.2 Análise do Produto

A tabela NCM raramente usa o nome comercial do produto. A segunda etapa pede ao modelo de linguagem que reescreva a descrição com o nome técnico usado pela NCM, em no máximo oito palavras e sem embalagem, medidas ou marca, e que liste as características que decidem a classificação, como material, função, aplicação e se o produto é completo ou uma peça. Quando algo importante não pode ser inferido da descrição, o modelo deve indicá-lo como "não informado". O *prompt*, reproduzido no Apêndice B, traz três exemplos de produtos que não fazem parte de nenhum conjunto de avaliação.

4.4.3 Busca de Candidatos

A busca é executada duas vezes, uma com a descrição original e outra com a descrição técnica, e as duas listas são combinadas por fusão de rankings (RRF, Equação 2.1, com $k = 60$), resultando em até 40 candidatos. A descrição original preserva termos que o modelo pode descartar indevidamente; a técnica traz o vocabulário da tabela.

Antes de consultar o índice, cada descrição passa por um tratamento: extraem-se apenas as palavras com três letras ou mais, o que também descarta números e medidas como "195/65" e "500ML"; removem-se palavras genéricas que aparecem em milhares de itens, como "tipo", "uso", "embalagem" e "unidade"; e cada palavra é buscada como prefixo, entre aspas, o que neutraliza caracteres e palavras reservadas da sintaxe de busca. A busca por prefixo cobre a maioria dos plurais, mas não os irregulares, como "farol" e "faróis"; para esses casos, o plural é gerado por regras simples de sufixo e acrescentado à consulta.

Quando uma palavra da descrição não encontra nenhum item na tabela, seus sinônimos são obtidos da OpenWordNet-PT, por meio da NLTK, e buscados em uma consulta separada, que ocupa uma pequena reserva de posições na lista. A consulta é separada porque, nos testes, misturar sinônimos de palavras comuns na consulta principal gerava ruído excessivo. Uma pequena lista manual complementa a WordNet em casos observados, como "azulejo", cujo termo oficial é "ladrilho".

4.4.4 Escolhas Restritas do Modelo

Com os candidatos em mãos, o modelo de linguagem faz duas escolhas curtas. Na primeira, os candidatos são agrupados por capítulo e apresentados em ordem de relevância, cada capítulo com seu título e alguns itens resumidos, e o modelo indica a qual capítulo o produto pertence ou responde “NENHUM”, se nenhum for do mesmo tipo de produto. Na segunda, o modelo recebe apenas os itens do capítulo escolhido, com o caminho completo de cada um, e devolve os três mais prováveis, em ordem, com uma justificativa curta, além da informação que falta para decidir, se houver. Essa informação é apresentada ao operador como uma pergunta, por exemplo “O valor Brix do suco de uva é superior ou igual a 30?”.

As duas respostas são restritas por esquema JSON (seção 2.5): o campo do capítulo só aceita os capítulos apresentados, e o campo de código só aceita os códigos apresentados. Assim, o modelo não consegue inventar um código inexistente nem escapar do formato esperado. O esquema da segunda escolha é ilustrado a seguir, com a lista de códigos abreviada:

```
{"type": "object",
 "properties": {
   "opcoes": {"type": "array", "minItems": 1, "maxItems": 3,
     "items": {"type": "object",
       "properties": {
         "justificativa": {"type": "string"},
         "codigo": {"type": "string",
                    "enum": ["8708.30.11", "8708.30.19", ...]}},
       "required": ["justificativa", "codigo"]}},
   "informacao_que_falta": {"type": "string"}},
 "required": ["opcoes", "informacao_que_falta"]}
```

O *prompt* de sistema que acompanha as duas escolhas (Apêndice B) orienta o modelo a pensar no que o produto é de fato antes de escolher, a não se deixar levar por palavras soltas como a cor, a evitar itens “Outros” por insegurança e, como desempate, a preferir a aplicação automotiva quando a descrição não diz o tipo de veículo, já que o ERP atende oficinas mecânicas.

Dois tratamentos adicionais se mostraram necessários. O primeiro é a tradução de referências cruzadas: o texto oficial cita outras posições apenas pelo número, como em “partes e acessórios dos veículos das posições 87.11 a 87.13”, e o modelo não tem como saber que essas

posições são motocicletas, bicicletas e cadeiras de rodas. Sem essa informação, pastilhas de freio de carro eram classificadas como peças de bicicleta; por isso, o nome curto de cada posição citada é acrescentado ao texto. O segundo é a validação da resposta: o Qwen2.5 eventualmente insere palavras em chinês no meio de respostas em português, e esses trechos são removidos; além disso, se a resposta vier fora do formato, a pergunta é repetida uma vez. Se o modelo devolver menos de três opções, a lista é completada com os próximos itens do capítulo na ordem da busca, identificados como alternativas sem análise do modelo.

4.5 EXECUÇÃO EM SEGUNDO PLANO E NOTIFICAÇÃO

Para que a sugestão já esteja pronta quando o operador a procurar, um processo em segundo plano consulta o banco a cada 20 segundos, identifica produtos com NCM inválido ainda sem sugestão e executa o processo de sugestão para cada um. O resultado é gravado em um cache SQLite local, separado do banco do ERP, cuja chave é o par código do produto e descrição: se o operador corrigir a descrição no ERP, a sugestão antiga deixa de valer e é recalculada. Ao concluir cada produto, o processo pode exibir uma notificação do Windows com a primeira opção sugerida.

Optou-se pela consulta periódica (*polling*) em vez de um gatilho (*trigger*) no banco porque um gatilho exigiria alterar a estrutura do banco do ERP em cada cliente, contrariando a premissa de não modificar o sistema principal. O custo é um atraso de até 20 segundos para o produto ser notado, além do tempo de sugestão.

4.6 INTERFACE DE SUPERVISÃO

A interface, desenvolvida em Streamlit, lista os produtos com NCM inválido e, para cada um, exibe as opções sugeridas, como ilustra a Figura 6. Acima das opções, são apresentadas a interpretação feita pelo modelo, as características consideradas, os produtos parecidos já cadastrados e, quando houver, a pergunta sobre a informação que falta. O operador escolhe uma das três opções e confirma; somente nesse momento o sistema executa a atualização do cadastro. Se nenhuma opção servir, o operador pode descartar a sugestão e pedir uma nova.

Figura 6 – Interface de supervisão com as opções sugeridas para um produto



Fonte: O autor (2026).

O exemplo da Figura 6 é um dos erros do conjunto de teste: para um refrigerante de cola, as três opções sugeridas são de águas e de bebidas fermentadas, e não a posição correta, de bebidas não alcoólicas (2202.10.00). É justamente para esse tipo de caso que a aprovação humana existe: o operador percebe que nenhuma opção corresponde ao produto e, em vez de aplicar a sugestão, pede uma nova ou classifica o produto manualmente.

A camada de acesso ao banco impõe duas restrições. Toda consulta de leitura passa por uma função que só aceita comandos `SELECT`. A gravação passa por outra função, que só aceita comandos `UPDATE` com parâmetros, e é chamada apenas pelo botão de aprovação da interface, com um comando fixo: `UPDATE PRODUTO SET NCM = ? WHERE CODIGO = ?`. Como o código gravado é sempre um dos códigos existentes na tabela, apresentados ao operador, não há como o sistema gravar um código inventado. Por controlar alterações no banco de produção, a interface é servida apenas para o próprio computador (*localhost*).

4.7 EVOLUÇÃO DAS VERSÕES

A arquitetura final resultou de uma sequência de versões, cada uma motivada pelos erros da anterior. O Quadro 2 resume essa evolução; os resultados quantitativos são apresentados no Capítulo 5.

Quadro 2 – Versões do processo de sugestão avaliadas

Versão	**Funcionamento**	**Situação**
Agente com ferramentas	LLM decide quais ferramentas chamar (busca na tabela, listagem de subcódigos, consulta ao banco) e em que ordem	Descartada: lenta, instável e com alucinações
Busca com *embeddings*	Busca semântica por vetores somada à busca lexical	Descartada: não aproximou o vocabulário comercial do técnico
Busca e escolha	Busca lexical e duas escolhas restritas do LLM; primeiro com uma opção, depois com três	Base da versão final
Navegação hierárquica	LLM navega capítulo, posição e item vendo a lista completa de cada nível	Descartada: mais lenta e menos precisa
Final	Busca e escolha, com análise do produto, pergunta ao operador, memória e nova tentativa	Adotada

Fonte: O autor (2026).

A primeira versão seguiu o projeto: um agente LangChain com o Llama 3.1 8B, que recebia ferramentas para consultar o banco do ERP e a tabela NCM e decidia, em várias rodadas, quais chamar. Na prática, cada classificação exigia de três a cinco chamadas sequenciais ao modelo, o que tornava a resposta lenta e expunha a instabilidade do servidor local sob uso repetido. Mais grave, o modelo nem sempre usava as ferramentas como instruído: em um dos testes, classificou um azulejo de banheiro como “tubo de escape”, código que não tinha relação com nenhum resultado de busca. Esse comportamento levou à decisão de inverter o controle: o programa executa a busca de forma determinística e o modelo apenas escolhe, com a resposta restrita aos candidatos.

Em paralelo, testou-se a busca semântica com dois modelos de *embeddings* (nomic-embed-text e paraphrase-multilingual-mpnet-base-v2), para encontrar itens sem palavras em comum com a descrição. Os vetores não aproximaram de forma útil o vocabulário comercial do técnico (“azulejo” de “ladrilho”, “bateria” de “acumulador”), e o uso de um segundo modelo obrigava o Ollama a alternar modelos na memória de vídeo a cada sugestão, o que travava o servidor. A expansão por sinônimos da WordNet, mais simples, foi adotada no lugar.

A versão de busca e escolha inicialmente devolvia uma única opção. A análise de erros mostrou que, quando o modelo errava, a resposta certa costumava estar entre os primeiros candidatos do mesmo capítulo (leite integral e desnatado, pilha AA e C), o que motivou a apresentação de três opções ao operador.

A navegação hierárquica foi testada para eliminar a dependência da busca: em vez de escolher entre candidatos, o modelo via a lista completa dos 96 capítulos, depois todas as posições do capítulo escolhido e, por fim, todos os itens das posições escolhidas, como faria um classificador humano seguindo as Regras Gerais de Interpretação. Apesar de acertar o capítulo com frequência, essa versão errou mais na escolha final e foi mais lenta, como mostra o Capítulo 5. A versão final retornou à busca e escolha, incorporando as melhorias que funcionam independentemente da forma de navegação: a análise do produto com características, a pergunta ao operador, a memória de classificações e a nova tentativa em caso de resposta fora do formato.

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados da detecção e da sugestão de NCM. Primeiro, relata-se a detecção; em seguida, a comparação entre modelos de linguagem, a comparação entre as versões do processo de sugestão e a análise dos erros da versão final. Ao final, os resultados são confrontados com as metas do projeto e discutidos, incluindo as limitações da avaliação.

5.1 DETECÇÃO

A detecção apontou todos os 33 produtos do conjunto de teste, cadastrados com NCM "00000000", bem como os produtos cadastrados manualmente durante os testes da interface, o que corresponde a uma taxa de detecção de 100%. Após a restrição aos itens cadastrados como mercadoria, nenhum serviço com código NBS foi apontado como erro; antes dessa restrição, um código NBS válido havia sido indicado incorretamente como NCM inexistente, o que motivou o ajuste. Esse resultado era esperado, pois a detecção é uma regra determinística sobre a tabela oficial, e não depende do modelo de linguagem. Seu alcance, porém, limita-se a códigos vazios, malformados ou inexistentes, conforme discutido no Capítulo 4.

5.2 COMPARAÇÃO ENTRE MODELOS DE LINGUAGEM

A comparação entre modelos foi feita no conjunto dev15, nas primeiras versões da arquitetura de busca e escolha, quando o sistema ainda devolvia uma única opção. A Tabela 2 apresenta o acerto de cada modelo na mesma versão do processo.

Tabela 2 – Acerto da opção única por modelo no conjunto dev15

Versão do processo	**Llama 3.1 8B**	**Qwen2.5 7B**
Busca e escolha, versão inicial	9/15 (60,0%)	11/15 (73,3%)
Busca e escolha, após ajustes no conjunto dev15	10/15 (66,7%)	13/15 (86,7%)

Fonte: O autor (2026).

O Qwen2.5 7B superou o Llama 3.1 8B nas duas versões, com tempo semelhante, em torno de 8 segundos por produto naquela versão do processo. Na inspeção das respostas, o Qwen2.5 demonstrou melhor compreensão das descrições em português e seguiu com mais consistência as regras do *prompt*, como desconsiderar a cor de um azulejo. Por esse motivo, ele

foi adotado nas versões seguintes. Seu principal defeito observado foi a inserção ocasional de palavras em chinês no meio das respostas, tratada por um filtro de caracteres.

Os números da Tabela 2, contudo, são otimistas: os ajustes da segunda linha foram feitos justamente observando os erros desses 15 casos. Quando a mesma versão que atingiu 86,7% no dev15 foi medida pela primeira vez no conjunto de teste, acertou a opção única em apenas 14 de 33 produtos (42,4%). Essa diferença ilustra a importância de separar os casos usados para ajustar o sistema dos casos usados para medi-lo, e motivou a criação do conjunto dev, com 41 produtos de 14 setores, para orientar as melhorias seguintes.

5.3 COMPARAÇÃO ENTRE VERSÕES DO PROCESSO DE SUGESTÃO

A Tabela 3 compara as três versões do processo que apresentam três opções ao operador, todas com o Qwen2.5 7B, nos conjuntos dev e teste.

Tabela 3 – Desempenho das versões do processo de sugestão com o Qwen2.5 7B

	dev (41 produtos)			**teste (33 produtos)**		
Versão	**1º lugar**	**Top-3**	**Tempo**	**1º lugar**	**Top-3**	**Tempo**
Busca e escolha	53,7%	61,0%	14,5 s	48,5%	63,6%	≈14 s
Navegação hierárquica	46,3%	63,4%	22,1 s	36,4%	54,5%	65,0 s
Final	**58,5%**	**70,7%**	**12,9 s**	**48,5%**	**63,6%**	**13,2 s**

Fonte: O autor (2026). Top-3: código correto entre as três opções apresentadas.

A navegação hierárquica acertou o capítulo em 90,0% dos casos do conjunto dev, contra 85,4% da busca e escolha, mas esse ganho não se converteu em acerto final: no conjunto dev, ela empatou no top-3 e perdeu no primeiro lugar, e, no conjunto de teste, perdeu nas duas métricas, levando ainda mais tempo. No teste, o tempo médio chegou a 65 segundos, valor possivelmente afetado pela instabilidade do servidor local observada ao longo do trabalho; no conjunto dev, a navegação foi cerca de 1,5 vez mais lenta que a busca e escolha e 1,7 vez mais lenta que a versão final. Uma variante que reservava uma vaga entre as três opções para cada posição diferente piorou ainda mais o resultado no dev (41,5% em primeiro lugar e 51,2% no top-3), porque a resposta certa costuma ser a segunda opção da mesma posição, como açúcar refinado e cristal.

A versão final melhorou o conjunto dev em relação à busca e escolha (de 53,7% para 58,5% em primeiro lugar e de 61,0% para 70,7% no top-3) e manteve exatamente o mesmo resultado no conjunto de teste, com um tempo ligeiramente menor. O acerto do capítulo na versão final foi de 90,0% no dev e 90,9% no teste. Como a versão final não piorou em nenhum

conjunto e acrescenta funcionalidades úteis ao operador, como a pergunta sobre a informação que falta e a memória de classificações, ela foi adotada.

5.4 ANÁLISE DOS ERROS DA VERSÃO FINAL

Na versão final, o código correto ficou fora das três opções em 12 dos 33 produtos de teste. A Tabela 4 classifica esses erros pelo nível da hierarquia em que ocorreram.

Tabela 4 – Erros da versão final no conjunto de teste, por nível da hierarquia

Tipo de erro	**Casos**	**Produtos**
Item errado na posição correta	6	sabonete, detergente, geladeira, óleo de motor, bola de futebol, lâmpada de farol
Posição errada no capítulo correto	3	refrigerante, cadeira de escritório, bateria automotiva
Capítulo errado	3	toalha de banho, notebook, furadeira
Total	**12**	

Fonte: O autor (2026).

Metade dos erros ocorreu no último nível: o sistema identificou corretamente a posição, mas não o item. São distinções finas, que dependem de uma informação ausente da descrição ou de um conhecimento sobre o produto que o modelo não aplicou: se o sabonete é de toucador, se o óleo de motor contém aditivos, se a lâmpada de farol é halógena ou de unidade selada, que a geladeira duplex é uma combinação de refrigerador e congelador e que a bola de futebol é inflável. No primeiro grupo de casos, a pergunta ao operador é o mecanismo mais adequado, pois o operador, com o produto em mãos, tem condições de responder; no segundo, o erro é do modelo, que dispunha da informação necessária na própria descrição.

Os erros de posição e de capítulo concentram-se em produtos cujo nome comercial está distante do texto oficial. O notebook foi classificado na posição de monitores e receptores de televisão, pois a NCM o descreve como “máquina automática para processamento de dados”; a bateria, como pilha, pois o termo oficial é “acumulador elétrico”; a cadeira, como móvel, pois a NCM a trata como “assento”; e a toalha de banho, como vestuário, pois a NCM a agrupa com roupas de cama e de banho. Em todos esses casos, a análise do produto não produziu o termo técnico que levaria a busca ao item correto, e o modelo escolheu o melhor entre candidatos errados. A expansão por sinônimos ajudou em casos como azulejo e ladrilho, mas não cobre essas relações, que são de vocabulário técnico, e não de sinonímia.

Destaca-se que três dos quatro produtos automotivos do conjunto de teste (óleo de motor, lâmpada de farol e bateria) estão entre os erros, enquanto, no conjunto dev, 10 dos 13

produtos automotivos tiveram o código correto entre as três opções. Os erros automotivos do teste são justamente os de distinção mais fina, o que sugere que o desempenho no segmento depende menos do setor e mais da especificidade da descrição.

A pergunta ao operador foi gerada em 4 dos 33 produtos de teste. Algumas foram pertinentes, como "O produto é descafeinado?" para o café, que separa dois itens da mesma posição; outras foram pouco úteis, como uma pergunta sobre o tipo de armazenamento para um celular. Em um caso, a pergunta reproduziu literalmente o exemplo dado no *prompt* ("A cadeira é de madeira ou de metal?"), o que indica que o exemplo deve ser trocado por um caso que não pertença a nenhum conjunto de avaliação.

5.5 ATENDIMENTO ÀS METAS DO PROJETO

O projeto estabeleceu como critérios de sucesso uma taxa de detecção de pelo menos 85% e uma taxa de acerto das correções de pelo menos 75%. A meta de detecção foi atingida, com 100% dos produtos com NCM inválido detectados. A meta de acerto não foi atingida: a primeira opção sugerida estava correta em 48,5% dos produtos de teste, e o código correto esteve entre as três opções em 63,6%.

Esses números devem ser lidos à luz do fluxo com supervisão humana. Em quase dois terços dos casos, o operador recebe, sem consultar nenhuma tabela, a classificação correta entre três opções explicadas, além do aviso sobre a informação que falta; nos 12 casos restantes, as opções pertenciam ao capítulo correto em 9, o que reduz a busca manual. Nenhuma sugestão continha código inexistente, já que as respostas são restritas aos códigos da tabela. O sistema, portanto, não substitui o conhecimento fiscal do operador ou do suporte, mas reduz o esforço de classificação e impede que o erro mais grave, o código inexistente, chegue à emissão da nota.

5.6 DISCUSSÃO

Três observações gerais emergem dos experimentos. A primeira é que restringir o papel do modelo foi mais eficaz do que ampliar sua autonomia. O agente com ferramentas, mais próximo da proposta original, produziu códigos sem relação com a busca e exigia várias chamadas por produto; a arquitetura de busca e escolha, em que o programa controla o fluxo e o modelo apenas escolhe entre candidatos verificados, eliminou a possibilidade de código inventado e reduziu o tempo e a instabilidade. Esse resultado converge com a recomendação da literatura de usar o LLM como ferramenta de proposição subordinada a mecanismos de

verificação e à decisão humana (Natarajan *et al.*, 2025; Huang *et al.*, 2025).

A segunda é que, com um modelo de 7 bilhões de parâmetros, o gargalo está na escolha fina, e não na orientação geral. O modelo acerta o capítulo em cerca de 90% dos casos, mas erra a escolha entre itens parecidos e não produz, com confiabilidade, o termo técnico que a tabela usa. A navegação hierárquica, que dá ao modelo mais decisões e listas mais longas, piorou o resultado por acumular erros ao longo dos níveis. Melhorias futuras tendem a vir mais da recuperação, de um vocabulário técnico mais rico e de informação adicional sobre o produto do que de mudanças na forma de navegar a tabela.

A terceira é metodológica: o resultado de 86,7% no conjunto usado para os ajustes caiu para 42,4% em casos novos. Sem a separação entre desenvolvimento e teste, o trabalho teria relatado um desempenho quase duas vezes maior do que o real.

5.6.1 Limitações e Ameaças à Validade

Os resultados devem ser interpretados considerando as seguintes limitações:

- **Tamanho dos conjuntos**: com 33 produtos de teste, cada produto equivale a cerca de 3 pontos percentuais, e diferenças pequenas entre versões não são estatisticamente conclusivas;
- **Gabarito**: a classificação correta foi conferida pelo autor na tabela oficial, e não por um classificador fiscal certificado; em produtos ambíguos, foram aceitos todos os códigos plausíveis;
- **Reuso do conjunto de teste**: o conjunto de teste foi medido uma vez para cada versão candidata e usado para decidir entre a navegação hierárquica e a busca e escolha, o que o torna levemente menos independente; uma medição definitiva exigiria um novo conjunto, medido uma única vez;
- **Dados fictícios**: os produtos foram cadastrados para os testes, e não extraídos de clientes; descrições reais tendem a ser mais abreviadas e ruidosas;
- **Memória não avaliada**: a base de desenvolvimento possui poucos produtos com NCM válido, de modo que o efeito da memória de classificações, que depende de um histórico de cadastros, não pôde ser medido;
- **Infraestrutura**: os tempos foram medidos em uma única máquina, e o servidor local de modelos apresentou instabilidades ao longo do trabalho, que podem ter afetado algumas

das medições de tempo.

6 CONCLUSÃO

Este trabalho investigou se é possível integrar um agente baseado em LLM a um banco de dados relacional para identificar inconsistências fiscais e propor correções de forma eficiente sob supervisão humana. Para responder a essa pergunta, foi desenvolvido e avaliado um protótipo integrado ao ERP Lux Auto, com foco na classificação NCM do cadastro de produtos, executado inteiramente em um computador convencional, sem custo de API e sem alterações no código ou na estrutura do ERP.

A resposta obtida é positiva, com ressalvas. A detecção de produtos com NCM inválido foi resolvida de forma determinística, com 100% de detecção no conjunto de teste, sem necessidade de modelo de linguagem. A sugestão da classificação, que é a parte que exige interpretação, apresentou o código correto em primeiro lugar em 48,5% dos produtos de teste e entre as três opções em 63,6%, em cerca de 13 segundos por produto, abaixo da meta de 75% estabelecida no projeto. Mesmo assim, o sistema nunca sugeriu um código inexistente, acertou o capítulo em 90,9% dos casos e entrega ao operador, junto com as opções, a justificativa e a informação que falta para decidir, o que torna a classificação manual mais rápida e informada.

Os objetivos específicos foram atendidos da seguinte forma: as inconsistências fiscais foram mapeadas e a NCM foi delimitada como foco; a tabela NCM foi estruturada em uma base local com busca textual; o agente foi projetado e implementado com detecção, sugestão, notificação e aprovação humana; foram construídos conjuntos separados de desenvolvimento e de teste; e o sistema foi avaliado quanto à detecção, ao acerto e ao tempo, com comparação entre dois modelos e entre arquiteturas.

As principais contribuições do trabalho são: uma arquitetura de busca seguida de escolha restrita, na qual o modelo de linguagem não acessa o banco nem pode responder com códigos inexistentes; a evidência experimental de que, com modelos pequenos, essa arquitetura supera tanto o agente com ferramentas quanto a navegação hierárquica nível a nível; e um protocolo de avaliação com conjuntos separados e métricas por nível, que mostrou o quanto a avaliação nos próprios casos de ajuste superestima o desempenho.

6.1 TRABALHOS FUTUROS

A partir dos resultados e das limitações observadas, sugerem-se as seguintes continuidades:

• **Demais categorias fiscais**: estender a arquitetura ao CST, ao CFOP e à relação entre Inscrição Estadual e Indicador de Contribuinte, cuja detecção pode, em boa parte, seguir regras determinísticas semelhantes às da NCM;

• **Detecção de código válido, porém incorreto**: comparar a descrição do produto com o texto do código cadastrado para apontar classificações existentes, mas incompatíveis com o produto;

• **Avaliação com dados reais**: medir o sistema, uma única vez, em um conjunto de produtos de clientes, anonimizado e com o gabarito conferido por um profissional de classificação fiscal, o que permitiria também medir o efeito da memória de classificações;

• **Vocabulário técnico**: construir, a partir das aprovações dos operadores, um dicionário de correspondências entre nomes comerciais e termos da NCM (“bateria” e “acumulador”, “notebook” e “máquina automática para processamento de dados”), atacando a principal causa dos erros de capítulo e posição;

• **Notas explicativas**: incorporar à busca as Notas Explicativas do Sistema Harmonizado, que descrevem o conteúdo de cada posição com exemplos de produtos;

• **Modelos maiores ou ajustados**: avaliar modelos maiores, quando o hardware permitir, ou um modelo pequeno ajustado com exemplos de classificação, comparando ganho de acerto e custo de execução;

• **Integração com o ERP**: substituir a consulta periódica por um aviso no próprio ERP no momento do cadastro, caso a desenvolvedora opte por essa integração.

REFERÊNCIAS

ARORA, Jitesh Kumar; ERTURK, Emre. **An Exploratory Study on the Implementation and Adoption of ERP Solutions for Businesses**. 2017. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/1701.08329.

BENDINELLI, Tommaso; DOX, Artur; HOLZ, Christian. **Exploring LLM Agents for Cleaning Tabular Machine Learning Datasets**. 2025. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2503.06664.

BIRD, Steven; KLEIN, Ewan; LOPER, Edward. **Natural Language Processing with Python**. Sebastopol: O'Reilly Media, 2009.

BRASIL. **Ajuste SINIEF 07/2005: Institui a Nota Fiscal Eletrônica e o Documento Auxiliar da Nota Fiscal Eletrônica**. 2005. Acesso em: 01 maio 2026. Disponível em: https://www.confaz.fazenda.gov.br/legislacao/ajustes/2005/AJ007_05.

BRASIL. **Emenda Constitucional nº 132, de 20 de dezembro de 2023. Altera o Sistema Tributário Nacional.** 2023. Acesso em: 09 jun. 2026. Disponível em: https://www.planalto.gov.br/ccivil_03/constituicao/emendas/emc/emc132.htm.

BRASIL. Receita Federal. **Nomenclatura Comum do Mercosul (NCM)**. 2026. Acesso em: 06 out. 2026. Disponível em: https://www.gov.br/receitafederal/pt-br/assuntos/aduana-e-comercio-exterior/classificacao-fiscal-de-mercadorias/ncm.

CHASE, Harrison. **LangChain**. 2022. Framework open-source para desenvolvimento de aplicações com LLMs. Acesso em: 01 maio 2026. Disponível em: https://github.com/langchain-ai/langchain.

CHENG, Edward; CHENG, Jeshua. **A Decoupled Human-in-the-Loop System for Controlled Autonomy in Agentic Workflows**. 2026. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2604.23049.

CHENG, Edward C.; CHENG, Jeshua; SIU, Alice. **Toward Safe and Responsible AI Agents: A Three-Pillar Model for Transparency, Accountability, and Trustworthiness**. 2025. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2601.06223.

CHIMPIRI, Tirumala Rao. Ensuring data quality in ERP implementations: Key conversion considerations. **International Journal of Advance Research in Computer and Communication Engineering**, v. 13, n. 3, 2024. Acesso em: 01 maio 2026. Disponível em: https://www.researchgate.net/publication/379352537.

CORMACK, Gordon V.; CLARKE, Charles L. A.; BÜTTCHER, Stefan. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *In*: **Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval**. ACM, 2009. p. 758–759. Acesso em: 06 out. 2026. DOI: 10.1145/1571941.1572114. Disponível em: https://doi.org/10.1145/1571941.1572114.

DOUGLAS, Michael R. **Large Language Models**. 2023. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2307.05782.

ENCAT; SEFAZ. **Manual de Orientação do Contribuinte: Padrões Técnicos de Comunicação da Nota Fiscal Eletrônica**. 7.0. ed. [S.l.], 2020. Acesso em: 01 maio 2026. Disponível em: https://www.nfe.fazenda.gov.br.

Firebird Project. **About Firebird**. 2024. Documentação oficial descrevendo o Firebird como fork open-source do InterBase 6.0. Acesso em: 01 maio 2026. Disponível em: https://firebirdsql.org/en/about-firebird/.

GHOLAMIAN, Sina *et al.* LLM-based robust product classification in commerce and compliance. *In*: **Proceedings of the 1st Workshop on Customizable NLP: Progress and Challenges in Customizing NLP for a Domain, Application, Group, or Individual (CustomNLP4U)**. Association for Computational Linguistics, 2024. p. 26–36. Acesso em: 09 jun. 2026. DOI: 10.18653/v1/2024.customnlp4u-1.3. Disponível em: https://doi.org/10.18653/v1/2024.customnlp4u-1.3.

GRATTAFIORI, Aaron *et al.* **The Llama 3 Herd of Models**. 2024. Acesso em: 06 out. 2026. Disponível em: https://arxiv.org/abs/2407.21783.

HONG, Zijin *et al.* Next-generation database interfaces: A survey of LLM-based text-to-SQL. **IEEE Transactions on Knowledge and Data Engineering**, 2024. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2406.08426.

HUANG, Lei *et al.* A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. **ACM Transactions on Information Systems**, v. 43, n. 1, p. 1–38, 2025. Acesso em: 01 maio 2026. Disponível em: https://doi.org/10.1145/3703155.

JURAYJ, William; HOLZENBERGER, Nils; Van Durme, Benjamin. Language models and logic programs for trustworthy tax reasoning. *In*: **Proceedings of the AAAI Conference on Artificial Intelligence**. AAAI Press, 2026. v. 40, n. 45, p. 38688–38698. Acesso em: 01 maio 2026. DOI: 10.1609/aaai.v40i45.41212. Disponível em: https://doi.org/10.1609/aaai.v40i45.41212.

KIM, Jaewoong; MIN, Moohong. From RAG to QA-RAG: Integrating generative AI for pharmaceutical regulatory compliance process. *In*: **Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing**. ACM, 2025. p. 1293–1295. Acesso em: 09 jun. 2026. DOI: 10.1145/3672608.3707749. Disponível em: https://doi.org/10.1145/3672608.3707749.

LAUKKANEN, Sanna; SARPOLA, Sami; HALLIKAINEN, Petri. Enterprise size matters: objectives and constraints of ERP adoption. **Journal of Enterprise Information Management**, Emerald, v. 20, n. 3, p. 319–334, 2007. Acesso em: 09 jun. 2026. DOI: 10.1108/17410390710740763. Disponível em: https://doi.org/10.1108/17410390710740763.

LEWIS, Patrick *et al.* Retrieval-augmented generation for knowledge-intensive NLP tasks. *In*: **Advances in Neural Information Processing Systems**. [s.n.], 2020. v. 33. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2005.11401.

LIU, Zechun *et al.* LLM-QAT: Data-free quantization aware training for large language models. *In*: **Findings of the Association for Computational Linguistics: ACL 2024**. Association for Computational Linguistics, 2024. p. 467–484. Acesso em: 09 jun. 2026. DOI: 10.18653/v1/2024.findings-acl.26. Disponível em: https://doi.org/10.18653/v1/2024.findings-acl.26.

MADICHARLA, Kalyan Pavan Kumar. Human-in-the-loop LLMOps: Balancing automation and control. **World Journal of Advanced Engineering Technology and Sciences**, v. 15, n. 2, p. 1092–1099, 2025. Acesso em: 01 maio 2026.

MEHTA, Rushabh. Beyond accuracy: Rethinking data quality as a strategic pillar in ERP implementation. **International Journal of Data Science and Machine Learning**, v. 5, n. 1, p. 400–409, 2025. Acesso em: 01 maio 2026.

MILLER, George A. WordNet: a lexical database for English. **Communications of the ACM**, v. 38, n. 11, p. 39–41, 1995. Acesso em: 06 out. 2026. DOI: 10.1145/219717.219748. Disponível em: https://doi.org/10.1145/219717.219748.

NATARAJAN, Sriraam *et al.* Human-in-the-loop or AI-in-the-loop? Automate or collaborate? *In*: **Proceedings of the AAAI Conference on Artificial Intelligence**. AAAI Press, 2025. v. 39, n. 27, p. 28594–28600. Acesso em: 01 maio 2026. DOI: 10.1609/aaai.v39i27.35083. Disponível em: https://doi.org/10.1609/aaai.v39i27.35083.

NAY, John J. *et al.* Large language models as tax attorneys: a case study in legal capabilities emergence. **Philosophical Transactions of the Royal Society A**, The Royal Society, v. 381, n. 2260, 2023. Acesso em: 01 maio 2026. Disponível em: https://doi.org/10.1098/rsta.2022.0327.

Ollama. **Ollama: Get up and running with large language models locally**. 2023. Acesso em: 01 maio 2026. Disponível em: https://ollama.com.

PAIVA, Valeria de; RADEMAKER, Alexandre; MELO, Gerard de. OpenWordNet-PT: An open Brazilian wordnet for reasoning. *In*: **Proceedings of COLING 2012: Demonstration Papers**. Mumbai: [s.n.], 2012. p. 353–360. Acesso em: 06 out. 2026. Disponível em: https://aclanthology.org/C12-3044.

PAN, Guanzhong *et al.* **A Cost-Benefit Analysis of On-Premise Large Language Model Deployment: Breaking Even with Commercial LLM Services**. 2025. Acesso em: 18 jun. 2026. Disponível em: https://arxiv.org/abs/2509.18101.

PÖHLER, Jan *et al.* Keeping the human in the loop: are autonomous decisions inevitable? **i-com**, De Gruyter, v. 24, n. 1, p. 9–25, 2025. Acesso em: 01 maio 2026. Disponível em: https://doi.org/10.1515/icom-2024-0068.

QU, Changle *et al.* Tool learning with large language models: A survey. **Frontiers of Computer Science**, 2024. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2405.17935.

Qwen Team. **Qwen2.5 Technical Report**. 2024. Acesso em: 06 out. 2026. Disponível em: https://arxiv.org/abs/2412.15115.

ROBERTSON, Stephen; ZARAGOZA, Hugo. The probabilistic relevance framework: BM25 and beyond. **Foundations and Trends in Information Retrieval**, v. 3, n. 4, p. 333–389, 2009. Acesso em: 06 out. 2026. DOI: 10.1561/1500000019. Disponível em: https://doi.org/10.1561/1500000019.

ROYCHOWDHURY, Aparup *et al.* Design and implementation of an AI agent for natural language-driven database operations. *In*: **2025 6th International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS)**. [S.l.: s.n.], 2025. Acesso em: 01 maio 2026.

Ruiz Gómez, Jorge *et al.* **Chatting with your ERP: A Recipe**. 2025. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2507.23429.

SCHELTER, Sebastian *et al.* Automating large-scale data quality verification. **Proceedings of the VLDB Endowment**, v. 11, n. 12, p. 1781–1794, 2018. Acesso em: 01 maio 2026. Disponível em: https://doi.org/10.14778/3229863.3229867.

SEFAZ-PE. **Nota Fiscal Eletrônica: Perguntas e Respostas**. [S.l.], 2016. Acesso em: 01 maio 2026. Disponível em: https://www.sefaz.pe.gov.br/Publicacoes/Manuais.

SHAO, Minghao *et al.* **Survey of different Large Language Model Architectures: Trends, Benchmarks, and Challenges**. 2024. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/2412.03220.

SINDIREPA BRASIL. **Anuário da Indústria de Reparação de Veículos do Brasil 2023**. 2023. Dados baseados na RAIS e CNAE 4520-0. Acesso em: 01 maio 2026. Disponível em: https://www.sindirepabrasil.org.br.

SQLite. **SQLite FTS5 Extension**. 2024. Acesso em: 06 out. 2026. Disponível em: https://www.sqlite.org/fts5.html.

Streamlit Inc. **Streamlit Documentation**. 2024. Acesso em: 01 maio 2026. Disponível em: https://docs.streamlit.io.

van Everdingen, Yvonne; van Hillegersberg, Jos; WAARTS, Eric. Enterprise resource planning: ERP adoption by European midsize companies. **Communications of the ACM**, ACM, v. 43, n. 4, p. 27–31, 2000. Acesso em: 09 jun. 2026. DOI: 10.1145/332051.332064. Disponível em: https://doi.org/10.1145/332051.332064.

VASWANI, Ashish *et al.* Attention is all you need. *In*: **Advances in Neural Information Processing Systems**. Curran Associates, Inc., 2017. v. 30, p. 5998–6008. Acesso em: 01 maio 2026. Disponível em: https://arxiv.org/abs/1706.03762.

WANG, Lei *et al.* A survey on large language model based autonomous agents. **Frontiers of Computer Science**, v. 18, n. 6, p. 186345, 2024. Acesso em: 01 maio 2026. Disponível em: https://doi.org/10.1007/s11704-024-40231-1.

WILLARD, Brandon T.; LOUF, Rémi. **Efficient Guided Generation for Large Language Models**. 2023. Acesso em: 06 out. 2026. Disponível em: https://arxiv.org/abs/2307.09702.

WORLD CUSTOMS ORGANIZATION. **What is the Harmonized System (HS)?** 2026. Acesso em: 06 out. 2026. Disponível em: https://www.wcoomd.org/en/topics/nomenclature/overview/what-is-the-harmonized-system.aspx.

XU, Huiying *et al.* Data quality issues in implementing an ERP. **Industrial Management & Data Systems**, Emerald Publishing, v. 102, n. 1, p. 47–58, 2002. Acesso em: 01 maio 2026. Disponível em: https://doi.org/10.1108/02635570210414668.

ZHOU, Zixuan *et al.* **A Survey on Efficient Inference for Large Language Models**. 2024. Acesso em: 09 jun. 2026. Disponível em: https://arxiv.org/abs/2404.14294.

APÊNDICE A – CONJUNTOS DE CASOS

Este apêndice lista os produtos dos três conjuntos utilizados na avaliação, com as descrições exatamente como foram fornecidas ao sistema e os códigos NCM aceitos como corretos.

Tabela 5 – Conjunto dev15

Nº	Descrição	NCM aceito(s)
1	vela de ignição	8511.10.00
2	amortecedor	8708.80.00
3	retrovisor	7009.10.00
4	radiador para automóvel	8708.91.00
5	para-choque	8708.10.00
6	PARACHOQUE	8708.10.00
7	farol	8512.20.11
8	embreagem	8708.93.00
9	pneu novo para carro de passeio	4011.10.00
10	correia dentada	4010.35.00, 4010.36.00
11	pastilha de freio	8708.30.19, 8708.30.11, 8708.30.90
12	disco de freio	8708.30.19, 8708.30.11, 8708.30.90
13	óleo lubrificante para motor	2710.19.31, 2710.19.32
14	bateria de chumbo para automóvel	8507.10.10, 8507.10.90
15	AZULEIJO DE BANHEIRO AZUL	6907.23.00, 6907.21.00, 6907.22.00

Tabela 6 – Conjunto dev

Nº	Descrição cadastrada	NCM aceito(s)	Setor
1	BOMBA D AGUA MOTOR	8413.30.90	automotivo
2	ALTERNADOR 12V	8511.50.10	automotivo
3	MOTOR DE ARRANQUE	8511.40.00	automotivo
4	PARABRISA DIANTEIRO	7007.21.00, 8708.22.00	automotivo
5	JUNTA DO CABECOTE	8484.10.00	automotivo
6	FILTRO DE OLEO	8421.23.00	automotivo

Nº	Descrição cadastrada	NCM aceito(s)	Setor
7	CABO DE VELA	8544.30.00	automotivo
8	EXTINTOR DE INCENDIO PO QUIMICO	8424.10.00	automotivo
9	BUZINA 12V	8512.30.00	automotivo
10	PNEU ARO 17 MOTO	4011.40.00	automotivo
11	CAMARA DE AR BICICLETA	4013.20.00	automotivo
12	FLUIDO DE FREIO DOT 4	3819.00.00	automotivo
13	ADITIVO PARA RADIADOR	3820.00.00	automotivo
14	ACUCAR REFINADO 1KG	1701.99.00	alimentos
15	OLEO DE SOJA 900ML	1507.90.11	alimentos
16	MACARRAO ESPAGUETE 500G	1902.19.00, 1902.11.00	alimentos
17	BISCOITO RECHEADO CHOCOLATE	1905.31.00	alimentos
18	SUCO DE UVA INTEGRAL 1L	2009.61.00, 2009.69.00	bebidas
19	AGUA MINERAL SEM GAS 500ML	2201.10.00	bebidas
20	VINHO TINTO SECO 750ML	2204.21.00	bebidas
21	PERFUME 100ML	3303.00.10	higiene
22	PAPEL TOALHA 2 ROLOS	4818.20.00	higiene
23	ESCOVA DE DENTES	9603.21.00	higiene
24	FRALDA DESCARTAVEL INFANTIL	9619.00.00	higiene
25	MEIA DE ALGODAO	6115.95.00	vestuario
26	CUECA BOXER ALGODAO	6107.11.00	vestuario
27	CHINELO DE BORRACHA	6402.20.00	calcados
28	FONE DE OUVIDO BLUETOOTH	8518.30.00	eletronicos
29	MICROONDAS 30L	8516.50.00	eletrodomesticos
30	LIQUIDIFICADOR 600W	8509.40.10	eletrodomesticos
31	VENTILADOR DE MESA	8414.51.10	eletrodomesticos
32	CHAVE DE FENDA	8205.40.00	ferramentas
33	ALICATE UNIVERSAL	8203.20.10	ferramentas
34	TRENA 5M	9017.80.10	ferramentas
35	TIJOLO CERAMICO 8 FUROS	6904.10.00	construcao
36	CANO PVC 100MM	3917.23.00	construcao
37	FIO ELETRICO 2,5MM	8544.49.00	construcao
38	LAPIS PRETO	9609.10.00	papelaria

Nº	Descrição cadastrada	NCM aceito(s)	Setor
39	SOFA 3 LUGARES	9401.61.00, 9401.71.00	moveis
40	BICICLETA ARO 29	8712.00.10	esporte
41	RACAO PARA CAES 15KG	2309.10.00	pet

Tabela 7 – Conjunto de teste

Nº	Descrição cadastrada	NCM aceito(s)	Setor
1	ARROZ BRANCO TIPO 1 5KG	1006.30.21	alimentos
2	CAFE TORRADO E MOIDO 500G	0901.21.00	alimentos
3	REFRIGERANTE COLA 2L	2202.10.00	bebidas
4	CERVEJA LATA 350ML	2203.00.00	bebidas
5	LEITE UHT INTEGRAL 1L	0401.20.10	alimentos
6	CHOCOLATE AO LEITE BARRA 90G	1806.32.10	alimentos
7	DETERGENTE LIQUIDO LAVA LOUCAS 500ML	3402.50.00	limpeza
8	SABONETE EM BARRA 90G	3401.11.90	higiene
9	SHAMPOO ANTICASPA 350ML	3305.10.00	higiene
10	CREME DENTAL 90G	3306.10.00	higiene
11	PAPEL HIGIENICO FOLHA DUPLA 12 ROLOS	4818.10.00	higiene
12	CAMISETA ALGODAO MASCULINA	6109.10.00	vestuario
13	CALCA JEANS FEMININA	6204.62.00	vestuario
14	TENIS ESPORTIVO	6404.11.00, 6402.19.00	calcados
15	TOALHA DE BANHO ALGODAO	6302.60.00	cama mesa banho
16	CELULAR SMARTPHONE 128GB	8517.13.00	eletronicos
17	NOTEBOOK 15 POLEGADAS	8471.30.12, 8471.30.19	eletronicos
18	GELADERA FROST FREE DUPLEX	8418.10.00	eletrodomesticos
19	LAMPADA LED 9W	8539.52.00	eletricos
20	PILHA ALCALINA AA	8506.10.19	eletricos
21	MARTELO DE UNHA CABO MADEIRA	8205.20.00	ferramentas

Nº	Descrição cadastrada	NCM aceito(s)	Setor
22	FURADERA ELETRICA 500W	8467.21.00	ferramentas
23	CIMENTO PORTLAND 50KG	2523.29.10	construcao
24	TINTA ACRILICA PAREDE 18L	3209.10.10	construcao
25	PARAFUSO SEXTAVADO ACO M8	7318.15.00	ferragens
26	CADERNO ESPIRAL 96 FOLHAS	4820.20.00	papelaria
27	CANETA ESFEROGRAFICA AZUL	9608.10.00	papelaria
28	CADEIRA DE ESCRITORIO GIRATORIA	9401.31.00, 9401.39.00	moveis
29	BOLA DE FUTEBOL	9506.62.00	esporte
30	FILTRO DE AR DO MOTOR	8421.31.00	automotivo
31	OLEO MOTOR 5W30 1L	2710.19.32	automotivo
32	LAMPADA FAROL H4 12V	8539.21.10	automotivo
33	BATERIA 60AH 12V	8507.10.90	automotivo

APÊNDICE B – *PROMPTS* DA VERSÃO FINAL

Os *prompts* foram escritos sem acentuação, como no código-fonte do protótipo.

PROMPT DA ANÁLISE DO PRODUTO

```
Voce conhece a Nomenclatura Comum do Mercosul (NCM). Para a
descricao de produto recebida (pode ter erro de digitacao),
responda:

- descricao_tecnica: o nome tecnico/formal que a NCM usa para esse
tipo de mercadoria. No maximo 8 palavras. Sem embalagem, volume,
peso, medidas, potencia, cor ou marca.
- caracteristicas: ate 15 palavras com o que decide a
classificacao: material, funcao, aplicacao (ex.: para automovel),
estado (novo, refinado, cru...), se e produto completo ou peca. Se
algo importante nao da para saber pela descricao, diga "nao
informado: ...".

Responda SEMPRE em portugues do Brasil, nunca em outro idioma.

Exemplos:
'TV LED 50 POLEGADAS' -> descricao_tecnica 'aparelho receptor de
televisao'; caracteristicas 'aparelho eletronico completo, tela de
LED, uso domestico'
'GUARDA ROUPA MDF' -> descricao_tecnica 'movel de madeira para
quarto de dormir'; caracteristicas 'movel completo, madeira
aglomerada (MDF), uso domestico'
'SUCO DE LARANJA 1L' -> descricao_tecnica 'suco de laranja nao
fermentado'; caracteristicas 'bebida nao alcoolica, fruta, pronto
para beber; nao informado: valor Brix'
```

PROMPT DE SISTEMA DAS ESCOLHAS DE CAPÍTULO E ITENS

```
Voce e um especialista em classificacao fiscal de mercadorias na
NCM (Nomenclatura Comum do Mercosul) da Receita Federal.
```

A descricao do produto vem do jeito que foi digitada no cadastro de um ERP: pode estar em maiusculas, sem hifen, abreviada ou com erro de digitacao (ex.: PARACHOQUE = para-choque de carro; AZULEIJO = azulejo). Os candidatos vem de uma busca automatica e CONTEM RUIDO: alguns nao tem nada a ver com o produto.

Regras:
1. Pense no que o produto E de fato (material, funcao) antes de escolher. Uma cor ou palavra solta no nome nao muda o que o produto e (um azulejo azul continua sendo ceramica, nunca peixe).
2. Se um candidato nomeia diretamente o produto (ex.: 'Para-choques e suas partes' para um parachoque), escolha ele.
3. Nao escolha um item 'Outros'/'Outras' so porque ficou em duvida: todo 'Outros' pertence a uma posicao especifica (ex.: 'Outros' de tijolos refratarios). Escolha primeiro a posicao que descreve o produto e so depois o item dentro dela.
4. Responda SOMENTE com codigos/capitulos da lista. Na escolha do capitulo, use NENHUM apenas se nenhum candidato for sequer do mesmo tipo de produto.
5. Contexto do negocio: o ERP e de uma oficina mecanica de automoveis. Quando a descricao nao diz o tipo de veiculo e o mesmo produto existir para automovel e para moto/bicicleta/trator/trem, prefira o de automovel. Isso vale so para desempatar -- um produto que claramente nao e peca de carro (ex.: azulejo) continua sendo classificado pelo que ele e.
6. Cada justificativa deve ser curta e em linguagem simples. Se a escolha exata depender de uma informacao que a descricao nao tem (ex.: tipo de ceramica, absorcao de agua), diga qual informacao falta -- e justamente por isso as outras opcoes existem.